From AI-Generated Code to Production Software

A practical playbook for building maintainable software with coding agents

Contents

Introduction: Getting It to Work Was Only the Beginning 1

Chapter 1: What Your Agents Have to Guess 5

Chapter 2: Choosing a Stack Is Not Enough 9

Chapter 3: The Cost of Early Versions 16

Chapter 4: When the Product Outgrows a Single Application 22

Chapter 5: Composing the Foundations Your Product Needs 27

Chapter 6: Designing How the Pieces Work Together 32

Chapter 7: Giving Agents the Context Their Role Needs 37

Chapter 8: Verifying Foundations Before Reusing Them 41

Chapter 9: Starting a Product on an Existing Foundation 45

Chapter 10: Evolving the Product Without Rebuilding Its Foundations 49

Conclusion: From Understanding to Your First Project 54

Sources and Further Reading 58

Introduction: Getting It to Work Was Only the Beginning

Some mornings, I would start at eight, open the conversation with my coding agent, and type one word:

Continue.

We had already prepared documents describing what to build. The agent would work, return with a result, and I would ask what came next. Then I would tell it to continue again. That cycle could occupy an entire working day. When I noticed a problem, I described it and asked for a correction.

The project was a small Python desktop application for managing employee information and displaying a few statistics. Its details are confidential, but the experience is familiar enough to explain without them. I knew Python and wanted to avoid learning Electron for that project. With an agent doing the implementation, I could see the application taking shape.

That was exciting. I could describe something I wanted and receive working software.

Keeping it maintainable was another matter.

The Application Worked. Changing It Was the Problem.

During the first two or three months of 2026, I encountered a similar pattern across several projects: a functional-programming package in Python, a command-line tool for Notion, and applications that did what I had asked but became painfully difficult for me to maintain.

On the Notion CLI, even a small problem could take two days to correct. The existence of a working version gave me little confidence about the cost of the next change.

If you have already built software with agents, you may recognize that situation. There is an application. It runs. You can demonstrate a feature. But understanding the implementation takes too much effort, corrections keep reopening old questions, and you hesitate before asking for another change.

Managing several projects makes the problem more visible. Each repository has its own arrangements. The same capability is implemented differently. Reviewing a new change means reconstructing decisions that should have been settled earlier.

My goal became broader than receiving another working implementation. I wanted good code that did what I asked, and a development process that let me keep improving the product.

The Work Around the Code

I started experimenting with agent-driven development in Cursor roughly two years before writing this playbook. Early on, I expected too much from the models and had too little process around them. I would ask for a change, become frustrated when it failed, and ask for more corrections.

As the tools and models improved, including my move to Claude Code, agents became more capable of producing the requested features. Yet important work remained difficult.

[An algorithm that has been stable for decades and does not depend on fresh external information can often be implemented from knowledge already represented in a model's training data. A technology integration is different. The correct API, configuration, or supported pattern can change between versions. The challenge is not that an algorithm is inherently easy or that a technology is inherently hard. It is that one task may depend on current, version-specific information while the other may not.]

An agent could produce a plausible integration from what it already knew. That knowledge did not necessarily match the dependencies in my project. Asking it to research the documentation helped, but the investigation itself still required judgment: which source mattered, which explanation fit the problem, and whether a proposed correction actually resolved its cause.

Initialization could become a long cycle of running commands, examining failures, researching, changing configuration, and trying again. Some of my own understanding was missing too. Selecting technologies I liked did not mean I knew how to integrate them well.

I spent months improving foundations through research, implementation, mistakes, and revision. The value of that effort became much clearer when I stopped repeating it for every product.

When I Could Finally Work on the Product

[One of the foundations I built during this period was a SaaS template: a reusable starting repository for a web application, with recurring integrations and conventions already prepared. It was one part of a broader effort to make common software foundations reusable.]

It took about two months of research and development to reach a base I was satisfied with. At times, investing so much effort in the starting point felt like making the project harder for myself.

Then I reused it.

I used the foundation for my own platform and for another tool focused on web research. Both experiences changed what starting a project meant to me.

I could work on the product.

I could clone the base, adapt the project information, supply the necessary environment values, handle the applicable deployment setup, and begin discussing the actual features. The integrations I had already prepared were there. I was no longer reopening the same foundational questions before doing useful product work.

This is the kind of speed I care about: reaching a usable starting point quickly, then being able to keep building on it. A foundation earns its place through the work it makes easier after initialization as well as during it.

Learning to Lead the Work

My review process also had to change.

Previously, I worked as though I were the only developer. Changes went onto the main branch, and I looked through the result. I did not consistently separate the work into features or use pull requests to define what I was reviewing.

[Moving to T3 Code, a tool for organizing coding-agent work in separate environments, supported a different way of working: agents could use their own worktrees and branches, with pull requests associated with specific needs.]

I increasingly acted as a tech lead.

Today, I discuss the need with an architect agent, examine a proposal, and ask a developer agent to explore the repository and understand the change. For substantial proposals, the developer raises questions before implementation. I pass those questions to the architect, who evaluates them and revises the document where necessary. We repeat that exchange until I understand the proposal and the developer has no remaining questions.

Implementation then produces a pull request. The reviewer, often the architect who prepared the proposal, examines the result. The developer applies corrections. Automated checks and my own review remain part of deciding whether the work is acceptable.

The developer still needs judgment. If the code reveals a problem with the proposal, I want it to explain the problem and bring it back for discussion. Clear direction should make execution easier while preserving that responsibility.

This process is still evolving. I do not consider it perfect, and the examples in this playbook describe how I work today, not a final operating model. Some changes made during implementation are not always reflected consistently in the proposal, and I still have work to do on capturing lessons from recurring mistakes. The process and the playbook will change as I learn what works and what does not.

What has already helped is organizing the work around defined responsibilities and reviewable changes. The aim is not to imitate an organization chart. It is to make decisions, implementation, and review easier to reason about.

Who This Playbook Is For

This playbook is for developers who can already build with agents and want stronger control over the result. You should be comfortable with Git, able to read TypeScript, and familiar with the basics of deployment. A SaaS or web application is the main reference point, although some principles apply to other kinds of software.

The first practical success is a deployed foundation. The next is a working product feature built on it. Those steps give you something concrete to inspect, use, and improve.

You remain responsible for deciding what the product should do. A prepared foundation can provide authentication, data access, API conventions, and other recurring capabilities. It cannot decide which problem your users need solved or whether your feature solves it.

My products follow that division. I am building DeesseJS to make reusable foundations and capabilities available so developers and their agents can spend more of their effort on the product. This playbook explains the thinking behind that work and gives you a framework for evaluating or building foundations of your own.

The practical starting point here is a new project on an existing foundation. An established codebase may benefit from the same principles, but adopting them can require its own migration decisions. Replacing an existing system deserves a separate assessment.

What We Will Examine

We will examine the decisions agents otherwise have to guess, the integration work left open by a technology list, and the cost of accepting early implementations without a suitable review process.

We will look at how applications can consume shared capabilities, how a minimal foundation can receive the blocks a product needs, and what contracts make those pieces work together. We will

also examine context, verification, starting a product, and preserving the benefits of reuse as it evolves.

Throughout, I distinguish what I have experienced from what I am still designing. A desired capability is not an existing feature, and one successful project does not establish that a foundation suits every use case.

You can apply this framework to your own foundations. You can also use it to judge whether existing DeesseJS products save work that your project would otherwise need to repeat.

The question that brought me here was practical: how could I keep the ability to produce software with agents while reducing the friction of maintaining and extending it?

Getting the application to work was the first milestone. Learning how to keep it changeable was the larger one.

Chapter 1: What Your Agents Have to Guess

A first version can be a real success and still leave you with a project you cannot confidently change.

I learned that distinction while building a small desktop application in Python to manage employees. I did not want to learn Electron, so Python looked like the quicker route. I could start in the morning, tell the agent to continue, and let it work through a long list of tasks. The application took shape. It ran. From the outside, that looked like progress.

But I could not maintain it. I did not understand how its pieces fit together well enough to change them with confidence. Similar projects followed the same pattern: a functional-programming package, a Notion command-line tool, and other applications that could demonstrate a feature but were difficult to evolve.

The important distinction is not whether an agent can produce code. It is whether the code and the surrounding decisions leave you with a system that can absorb its next change.

When "It Works" Is Too Early

A running feature answers one question: did this implementation produce the behavior I requested in the situation I tried?

It does not answer several other questions:

- Can I explain where the behavior lives?
- Can I change it without accidentally changing another capability?
- Does it follow the versions and conventions the project actually uses?
- Does it fail safely when an assumption is false?
- Can another developer or agent find the right extension point?
- Will the project still be understandable after ten more features?

These questions are easy to defer because the first demonstration rewards visible output. The implementation may render a screen, save a record, or return a response. Hidden costs appear later, when a related feature exposes a missing boundary or the original implementation must be changed.

This is why "it works" is an incomplete acceptance test. It describes an observed outcome, not the conditions under which the outcome remains reliable. A prototype may be valuable even when it is disposable. The risk begins when a prototype is treated as a foundation without examining what it assumes.

My early Python desktop app showed the difference. The agent could keep making progress while I typed "continue." But when the project needed a change, I could not reason about the structure well enough to tell whether a proposed fix solved the right problem. The agent's ability to keep producing code concealed the fact that I had little control over the resulting system.

A useful question is therefore not only, "Can the agent build this?" It is, "What must be true for the next person or agent to change it safely?"

A Short Request Contains Several Kinds of Decisions

Suppose you ask an agent to add employee invitations to a web application. The visible request sounds small: an administrator enters an email address, and the employee receives an invitation.

Yet the behavior crosses several concerns. The application needs to decide who may invite someone, how invitation tokens are created and expire, where invitation state is stored, how delivery is requested, what the public link does, and what happens when a token is invalid or already used.

The request also depends on existing project decisions. Does the dashboard call a server API, or does it import a database client? Is there already an email capability? What identity model does authentication use? Are there existing conventions for validation, errors, logging, and tests? Which package owns invitation rules?

Some of these are product decisions. For example, does an invitation expire after a set period, and can an administrator revoke it? Others are integration decisions. Which API endpoint accepts the operation, and what is the existing way to trigger an email? Still others concern architecture and quality: where should the rule live, which components are allowed to depend on it, and what verifies that expired invitations are rejected?

A task can leave each of these decisions open without looking ambiguous in natural language. The agent then has to infer the missing information from the code, its training, or a search. It may make a defensible choice. It may also make a choice that conflicts with the product or system.

The useful unit of analysis is not the sentence in the prompt. It is the set of decisions and assumptions that sentence activates.

Four Gaps Can Look Like One Agent Mistake

When an implementation goes wrong, it is tempting to call the cause "the agent made a mistake." That may be true at the surface level, but it is not enough to decide how to prevent a recurrence. Several different gaps can produce similar symptoms.

Gap	What is missing	Example in the invitation feature
Product gap	The intended behavior is undecided	Should an invitation be reusable or single-use?
Knowledge gap	Current or project-specific information is absent	Does the installed email package support the method the agent chose?
Architecture gap	Ownership and dependency boundaries are unclear	Should the dashboard access the database directly?
Verification gap	There is no reliable check for a failure mode	Is there a test for an expired invitation?

These categories matter because they need different responses. A product gap needs a decision from the product owner. A knowledge gap needs research in sources appropriate to the dependency version. An architecture gap needs an explicit boundary or contract. A verification gap needs a check that can detect the unwanted behavior.

Treating all four as prompt-writing problems creates long instructions that may still fail to answer the actual question. Treating all four as coding defects sends agents into repair loops without resolving the underlying uncertainty.

The categories also interact. An unclear product rule can create a weak architecture proposal. A weak proposal can leave the developer unsure which package owns the behavior. The implementa-

tion may then pass a test that never covered the disputed behavior. The final result looks like one bad implementation, but the uncertainty entered much earlier.

How a Plausible Guess Makes the Next Change Harder

The most expensive guesses are often plausible. A developer can choose a familiar library, add an endpoint, or place a function in a nearby package. The code may pass the current scenario while quietly creating a dependency that limits the next one.

Imagine the invitation behavior is implemented in the dashboard and writes directly to the database. That might work for the web dashboard. Later, the product adds a command-line client, a mobile application, or an external SDK. Each client now needs to duplicate the invitation rules or reach into database details it should not own.

The first implementation saved a boundary decision. It did not remove the decision. It moved the cost to every future client.

This is how local convenience becomes system-level friction. Repeated implementations drift. A token rule changes in one client but not another. A retry creates duplicate emails. A schema migration has to coordinate with several applications that assumed direct access to the same tables.

The point is not that every feature needs a large architecture exercise. It is that some seemingly small choices determine where future changes must be made. If a choice is likely to be repeated or relied on, make it visible before it becomes a convention by accident.

In my own projects, this problem became especially painful when several technologies had to work together. The initial code could look reasonable, but one incorrect assumption about authentication, data access, or package structure caused new errors elsewhere. The agent tried to correct the visible symptom and sometimes introduced further changes. Progress stopped because the system-level question had not been answered.

What a Foundation Can Remove From the Problem

A prepared foundation can settle recurring technical decisions before a product feature needs them.

In the invitation example, a suitable foundation might already establish that applications call a server API, the server owns database access, authentication provides a known identity context, and email is requested through a shared capability. It might include conventions for schema changes, API input validation, error responses, and test placement.

That does not decide the invitation product behavior. The owner still has to specify expiry, revocation, duplicate invitations, roles, and user-visible messages. A prepared API and email integration cannot determine what the product should do.

The foundation reduces the number of fresh technical decisions. It does not reduce the need for product judgment, and it cannot make an incompatible requirement fit without discussion.

This distinction is central to the playbook. DeesseJS is being developed to provide reusable foundations and capabilities for common technical work. The intended benefit is that developers and agents can begin from known integrations and spend their attention on the behavior particular to their product. Whether a specific available foundation does that for a project still needs to be checked against its actual features and support conditions.

A foundation is useful when it makes important decisions easier to inspect and more consistent to reuse. It is harmful when it hides decisions that a project needs to make differently.

Identify the Decisions Your Next Task Leaves Open

Choose a feature you want to build, such as sending an invitation, adding a billing page, or importing a CSV. Before asking an agent to implement it, create a small decision map.

For each concern, write one of four things: decided, needs research, needs an architectural boundary, or needs a verification check. If a question has no known answer, do not disguise it as an implementation detail.

Here is a worked example for an invitation flow:

Concern	Current answer	Evidence or next action
Who can invite?	Workspace administrators	Confirm product permissions
Can an invite be reused?	No, one successful acceptance	Include in product behavior and tests
Who stores invitation state?	Server-side database package	Confirm existing API boundary
How is email sent?	Existing mail capability	Check current package documentation and integration
How does the dashboard start the flow?	Call the server RPC client	Trace another authenticated mutation
What happens after expiry?	Reject and allow a new invite	Define response behavior and test case
What must never happen?	A client must not write invitation rows directly	Review architecture convention and enforce if possible

This map is deliberately more concrete than a prompt such as “implement invitations securely.” It makes clear what is already decided, what evidence supports the choice, and where the agent should return with a question.

Now ask which answers belong in the reusable foundation and which belong to this product. “Dashboard calls the server API” may be a project-wide convention. “Invitation expires after seven days” is likely a product rule. “An expired token must not be accepted” is behavior that should be testable. “Use the latest recommended invitation library” is not a decision until current documentation and compatibility have been checked.

You can use the following sequence to decide whether to proceed:

1. If the desired behavior is unclear, return to product discovery.
2. If the technology behavior is uncertain, research the installed version and relevant primary documentation.
3. If ownership or dependency direction is uncertain, describe the affected components and their contracts.
4. If the intended behavior can be stated but not reliably checked, define a test, CI rule, or manual review observation.
5. Once the important decisions are visible, hand the task to implementation and preserve room for the developer to flag a contradiction.

The exercise is complete when the task no longer depends on the agent guessing the consequential parts. It does not require predicting every future change. It creates enough shared understanding to make the first implementation reviewable.

Chapter 2: Choosing a Stack Is Not Enough

I Thought a Few Days Would Be Enough

When I started building a desktop application for someone, I knew which technologies I wanted to use. Electron for the application, TanStack and shadcn/ui for parts of the interface, and Drizzle for data access. I understood their roles and had enough experience to choose them.

I expected the project to move quickly. With agents writing the code and tools whose value I understood, a few days seemed reasonable.

Work on the foundations eventually stretched from roughly February to the end of May, almost every day.

Much of that time went into understanding how to integrate the technologies. Some obstacles could stop progress for days. For one database-related problem, I spent three or four eight-hour days working with an agent that was trying to find a satisfactory fix. Partial fixes introduced other problems. Understanding the issue did not advance at the same pace as changes to the code.

The breakthrough came after I took the time to research the problem in depth. I was able to bring in information that had been missing from the investigation. Once that point was understood, the other work could resume.

This account does not establish that agents caused the entire delay. My own learning, the application's requirements, and integration decisions also mattered. It does, however, illustrate a practical limitation: having an agent that can modify code does not guarantee that the knowledge needed to resolve a problem is available.

> [**Editorial note:** the precise database issue is not necessary to the chapter's point. This example illustrates that deeper research unlocked progress, without attributing the delay to a specific technical cause].

A Technology List Leaves Decisions Open

A stack describes the tools selected for a project. It does not describe the system they will form.

Consider a list such as Electron, React, an RPC framework, an authentication library, a relational database, an ORM, and a component library. The list names the pieces. It does not say which part owns a user's identity, which code may read or write data, how a request crosses from the application to the server, or what happens when one step fails.

The missing information lives in the relationships between technologies. A database and an ORM each have documented APIs, but their integration still depends on how the project models data, runs migrations, creates connections, and exposes operations. An authentication provider may establish a session, but the application still needs a consistent way to enforce permissions where data is accessed. A UI component library can supply a form control, but it cannot decide which validation belongs in the browser, the server, or both.

None of those questions is answered by selecting a popular package.

Follow one user action through the system. Imagine an organization owner inviting a colleague from a dashboard. The dashboard collects an email address and sends a request through the project's RPC client. The server authenticates the caller, checks whether that person can invite members, applies the product's invitation rules, writes the invitation through the data layer, and asks a mail capability to send the message. The invited user later opens the public application, follows a link, and accepts the invitation through a separate request.

That feature touches more than a screen and a table. It crosses application, network, identity, authorization, business logic, persistence, and email boundaries. Each boundary raises a different question:

- Which application is allowed to initiate this action?
- Which server operation represents the action?
- Where is the permission check enforced?
- Which capability owns invitation rules?
- What is the authoritative record of membership?
- How are failed or repeated requests handled?
- Which service sends the email, and what happens if delivery fails?
- Which checks show that the whole flow behaves as intended?

The answers depend on product requirements and system design. The agent can investigate the existing code and propose how to connect the pieces. It should not invent consequential decisions because the request sounded straightforward.

A compact map can make the system visible:

Concern	Decision the project needs	Evidence that helps
Application	Which client owns the user interaction?	Product flow and application boundaries
API	Which operation represents the request?	A defined contract and existing RPC conventions
Identity and access	How is the caller identified, and where is access checked?	Authentication behavior and permission rules
Data	Which model is authoritative, and who can change it?	Schema, data ownership, and migration behavior
Integrations	Which capability sends mail or performs another side effect?	Current provider documentation and the project's adapter
Failure and retry	What happens when a request is repeated or one step fails?	Explicit behavior and targeted tests
Delivery	Which runtime receives the code and configuration?	Deployment setup and environment requirements

This is not a required architecture for every project. It is a way to see that a list of tools does not explain the path a real operation takes through them.

The Integration Graph Is the Work

For each technology, a project has to decide what it owns and how other parts use it. When several technologies are combined, those relationships form an integration graph.

The graph is not just a diagram of packages. It includes contracts, data flow, identity, configuration, operational behavior, and permitted dependencies. A project might decide that a browser application calls the server through an RPC client, that the server performs authorization, and that only

server-side packages access the database. Those are meaningful rules because they describe which direction a request travels and where responsibility belongs.

A second project could make different choices and still be coherent. The important thing is that the decisions fit the requirements and are applied consistently.

If boundaries are missing, agents can create locally plausible implementations that disagree with each other. One feature may call the database directly because the ORM is available in the application workspace. Another may call the API. One package may treat the authentication provider as the source of authorization, while another checks a separate role table. Both can compile. A user may even complete a happy path. Yet the system now contains competing ways to express the same responsibility.

That inconsistency raises the cost of the next change. Someone has to discover which path is correct, decide whether the new feature should follow it, and determine whether the older code should be reconciled. An agent may copy the nearest example, even when that example is the exception rather than the rule.

This is why integration design matters to maintainability. The issue is not whether the repository uses a monorepo or a particular number of packages. The issue is whether responsibilities and permitted relationships remain understandable as the product changes.

Constraints We Do Not Yet Know to Look For

Some questions are visible from the beginning. Others emerge only as the project develops.

A particular difficulty comes from knowledge we do not yet realize we need. We may know that a technology serves a purpose while overlooking a constraint that will matter during deployment, the first change, or the addition of another application.

This creates a research problem. We usually ask an agent to solve difficulties we have identified. If we are unaware of a constraint, it may be missing from both the request and the checks we perform.

Asking for web research can improve an answer, but it does not guarantee sufficient investigation. Documentation must match the version and context in use. A plausible solution still needs to be evaluated against the project. Some options become relevant only after requirements have been clarified.

At least three activities deserve separate attention: finding information, understanding its scope, and verifying its application. An answer backed by documentation may still be incomplete in the latter two respects.

This uncertainty never disappears entirely. The goal is to reduce consequential assumptions and organize the work so that constraints are discovered before they become expensive.

Familiar Code and Current Integration Knowledge Are Different

In my own work, self-contained algorithms and data structures were often easier to delegate than technology integrations. An integration depended on documentation, supported versions, configuration, and relationships between packages that the agent did not always understand adequately.

This is an observation about my projects, not a claim that algorithmic work is always correct or that every integration is difficult. It explains why I put so much effort into preparing integrations once and keeping their usage discoverable. The agent should be able to build on that knowledge in the next project.

Selecting a foundation starts with the kind of application and its requirements. Familiarity with its technologies can help you inspect and maintain it, but familiarity alone does not establish that the integration fits the product.

Code Does Not Automatically Explain Its Rationale

Reading code shows what has been implemented. It does not always explain why that implementation was chosen, which constraints it satisfies, or which changes might undermine it.

This distinction matters for both developers and agents.

An agent may reuse a familiar example to produce an initial version. It may also follow a pattern already present in the repository. But if the reasons behind that pattern remain implicit, a later request may lead it to bypass an important decision to meet a local requirement.

"Senior patterns" is too vague to serve as an instruction. It needs to be translated into responsibilities, interfaces, rules, and verifiable criteria. Specifying where a check belongs, which data may cross a boundary, or which package owns a responsibility gives more usable guidance than a general request for quality.

Documenting these decisions also makes them easier to revisit. A convention that was useful in one context may become insufficient in another. Its rationale helps distinguish a rule that remains necessary from a habit retained without a reason.

Microsoft's Azure architecture guidance recommends recording decisions that affect system structure, important quality attributes, or are difficult to reverse. AWS guidance similarly describes capturing the context and consequences of significant decisions, including decisions about dependencies and interfaces. This supports recording the reasoning that shapes a system. It does not mean every small implementation choice needs a formal architecture document.

The document's location should follow its purpose and readers. In my workflow, product reasoning and architecture proposals belong in an external decision workspace. The developer agent receives the decisions and task context it needs to implement. Repository documentation is for durable instructions developers need while working in that repository, such as package conventions and commands. A historical discussion about a feature is not automatically useful context for every future coding task.

The goal is not more documentation. It is to put the right decision where the right role can use it, and to avoid making an implementation agent reconstruct important context from old code or a long decision history. (Microsoft Azure Well-Architected Framework, Architecture Decision Records, AWS Prescriptive Guidance, ADR Process)

When Fixes Stop Moving the Project Forward

In my Electron project, the agent could continue changing code while our understanding of the obstacle remained inadequate. A partial fix introduced other difficulties. The project changed, but the underlying problem persisted.

A long session does not by itself mean the approach is wrong. Some investigations require several experiments. A more useful warning is that each attempt expands the implementation while the hypothesis remains vague. If the agent cannot explain what the next change is testing, and the last change has not established what was learned, another patch may make the system harder to reason about.

At that point, pause the sequence of edits. State the expected behavior, the observed behavior, and the specific question the next experiment should answer. Then choose the smallest useful way to

test it: inspect the relevant documentation, reduce the integration to a minimal reproduction, or compare two approaches against the same requirement.

For example, if a database operation works in one runtime but fails in another, first establish where the code is running and which client is being used. Then check the version-specific guidance for that runtime and create the smallest test that distinguishes a configuration problem from a data-model or transaction issue. This does not explain my Electron incident. It shows how to make an investigation concrete.

Choose a change that can distinguish between plausible explanations. If it cannot, the result may add activity without useful knowledge. After each attempt, record the observation in the task or return it to the architect so the next proposal reflects what has been learned.

This process does not promise an immediate answer. It turns experiments into evidence and limits changes whose consequences become difficult to understand. Research is useful when it changes what we know, not merely when a search returns a plausible explanation.

One Application Can Become Several

Integration decisions become even more important as the product expands.

A web application may be joined by a public API, a command-line tool, an MCP server, an SDK, or a desktop application. Each has its own constraints. They may also need to share capabilities and remain compatible.

Adding a client can raise authentication questions. Publishing an SDK requires examining exposed contracts. Changing a shared capability can affect several interfaces. A decision that was acceptable for one application may need to be reconsidered when the product includes several.

You do not need to build all these surfaces on day one. You need to distinguish current needs from plausible extensions, then choose boundaries that allow the product to evolve without imposing a disproportionate architecture.

The agent must be able to understand those boundaries. Otherwise, it may treat each request as an independent problem and create multiple implementations of capabilities that should be shared.

In my architecture, applications are clients. They use the server API and the appropriate authentication client. They do not access the database directly. That separation lets a dashboard, a public web application, and later clients use shared server capabilities through their own client interfaces.

This is a design choice, not a universal law. A small application may have no need for every boundary. Adding layers without a requirement can increase work rather than reduce it. The useful question is whether a boundary provides a capability the product needs, such as independent clients or a stable public API, and whether its cost is justified now.

The 2025 DORA report offers a useful external check. Its survey of nearly 5,000 technology professionals found a positive relationship between AI adoption and delivery throughput, while also reporting a negative relationship with delivery stability. DORA describes AI as an amplifier of the system around it and points to foundations such as internal platforms, fast feedback, and architecture that allows teams to work independently. These are reported associations, not proof that a particular architecture or template will produce the same outcome in an individual project. They reinforce a practical question: can the surrounding system safely absorb the additional code and changes an agent can produce? (DORA, State of AI-assisted Software Development 2025)

Turning Research into Reusable Foundations

The same learning process accompanied the development of my web template. Roughly two months of research and development were needed to arrive at a satisfactory base. Early versions still required substantial revision.

At times, I considered a version highly developed, then returned three weeks later with new knowledge and identified several improvements. Over time, decisions stabilized and the changes needed became smaller. The broader maturation of my foundations took close to a year.

A reusable base preserves part of the result of that journey. It contains code, integrations, and conventions. It can also contain references and explanations that help agents work within those decisions.

Today, starting from an existing template is much quicker for me. I clone the repository, explain the project and the origin of the base to the agent, then adapt details such as the README and package metadata. I configure the necessary environment variables and deployment. Product-specific features still need to be built afterward.

The preparation happened earlier. Reusing it reduces the number of decisions and integrations that need to be revisited for the new project.

That value depends on how well the base fits the need. An integration that works in one context is not universal. Tool versions, deployment constraints, and expected capabilities must remain understandable. The base must continue to be maintained and verified.

Reuse Changed the Starting Point

I reused the SaaS foundation for my own platform and for another tool focused on web research. What felt different was straightforward: I could work on the product. Previously, starting a project meant spending substantial time initializing it and correcting integration problems.

These are personal examples, not a controlled comparison. Their significance is the transfer of completed preparation into another project. The next feature still needed design and review, but the starting point no longer required the same assembly work.

Trace One Real Flow Through Your Stack

Before handing integration work to an agent, choose one real user action and trace it through the system. The invitation flow is one example. Your feature might instead involve signing in, uploading a file, sending a notification, or creating a subscription.

Part of the flow	Question to answer
User interaction	Which application owns this step, and what does the user see?
Request	Which existing client and server contract carry the action?
Identity	How does the server establish who is making the request?
Permission	Where does the system decide whether this user can do it?
Business rule	Which capability owns the behavior and its edge cases?
Data	Which model is authoritative, and which operation changes it?
Side effect	How are email, storage, or another external service used?
Failure	What happens when a request is repeated or one step fails?
Evidence	Which tests or observable behavior show that the intended flow works?

Not every feature needs a document answering every row. The exercise is useful when it reveals assumptions that would otherwise be left to the implementer. Some answers already exist in a project capability or contract. Some require product judgment. Others require research against current documentation. If an answer remains uncertain, make that uncertainty visible before it becomes a hidden implementation decision.

A concise architecture proposal can present the affected applications, packages, API operations, data models, and external providers, together with important choices and sources. It can show what the feature touches without prescribing a rigid sequence of code edits. The developer can then inspect the repository for task context and implement against the agreed boundaries.

You do not need to master everything before starting. You need to make important decisions accessible, acknowledge uncertainty, and progressively verify what you build.

What Your Stack Needs to Become

A technology list becomes a foundation when the relationships between its components are designed, documented, and verified in a given context.

For work with agents, this preparation provides a more explicit starting point. It reduces the decisions they must improvise and helps the developer understand proposed changes.

Work completed once can then serve subsequent projects. Reusing that work makes a fast start possible without assuming that every integration difficulty has disappeared.

Chapter 3: The Cost of Early Versions

First draft of the playbook "From AI-Generated Code to Production Software."

Give Each Need a Reviewable Change

Earlier in my agent workflow, changes went onto the main branch and I examined the result. I worked as though I were developing alone, without consistently separating features into pull requests.

Moving to T3 Code supported separate worktrees and a pull-request workflow. Each agent could work in its own environment and branch, then submit a change for review. That changed how I assigned work. Instead of asking an agent to keep adding unrelated code to the shared project, I could give it a defined need and review the result as a tech lead.

The useful boundary is a change whose purpose and effect can be understood. One pull request per need is a practical starting point, but it does not mean one pull request per file or one arbitrarily tiny change at a time. If several pieces are required for a complete behavior, the proposal and pull request should make their relationship clear. If preparatory changes can be reviewed independently, separating them may make the final feature easier to understand.

Google's engineering guidance recommends focused, self-contained code reviews because they can make a change's purpose and likely defects easier to examine. It also recognizes that some changes cannot be divided safely, such as broad refactors or features that must land as a complete unit. (Google Testing Blog, "In Praise of Small Pull Requests"; Google Engineering Practices, "Small CLs")

A large empirical study adds an important qualification. Kudrjavets, Nagappan, and Rastogi analyzed 845,316 pull requests across 100 actively developed GitHub projects and found no relationship between PR size or composition and time to merge. Their additional analysis of code reviews from Gerrit and Phabricator supported the same result. That finding does not mean every change is equally easy to understand. It means that reducing line count alone should not be sold as a reliable way to make changes merge faster. (Kudrjavets, Nagappan, and Rastogi, 2022)

For my workflow, the reason to organize a pull request around one need is review clarity: I want to understand the purpose, affected system areas, and evidence without unrelated changes obscuring them. A coherent feature may still touch many files. Artificially splitting tightly related work can make its behavior harder to evaluate. Choose a scope the reviewer can reason about, not a target line count.

A large feature can still need smaller changes. The review unit should let someone understand what the change is for, what it affects, and what evidence supports it. Generating more code should not make the object of review indefinitely larger.

The First Version Opens the Review

When an agent produces a first implementation, I treat it as something to examine. It provides a concrete result we can compare with the proposal, inspect in the code, and try in the application.

That distinction matters because generating a version and deciding whether it is suitable are different activities. The implementation can satisfy part of the request while still introducing unnecessary complexity, placing responsibilities in the wrong packages, or reflecting a misunderstanding in the original proposal.

In my current workflow, I try to resolve many of those issues before doing my own detailed review. The developer produces a version, a separate agent reviews it, and I relay the findings back to the developer. They iterate until the reviewer considers the result satisfactory. I then examine it myself.

This process grew out of experience with early versions that required substantial revision. The question became how to organize that revision so it produces a clearer, more maintainable result rather than an expanding collection of patches.

Separate the Decisions from the Coding

I increasingly use a more capable model for the technical proposal and for review, while a less capable model performs the implementation. The developer's role is becoming more focused on writing code within decisions established earlier.

That is how I currently allocate work. It does not establish that one model configuration is optimal for every project. The useful distinction is between responsibilities: understanding and proposing the change, implementing it, and examining the result.

The architect agent first explores the repository and understands the relevant parts of the system. It prepares a proposal that helps me see what the feature will touch and what it will enable. If the feature changes the database, I want to understand the affected schema. If it adds an API operation, I want to see the route or contract. If it changes the dashboard or another application, I want those affected surfaces to be visible. I like seeing sources when the proposal depends on technology-specific behavior.

This proposal is not a rigid list of edits for the developer to follow line by line. It is an impact map and a place to examine product, architecture, and integration decisions before the code exists. The implementation agent still inspects the repository in its own worktree so it can understand the current code and the proposal in context.

Before implementation, I often ask the developer agent whether it has questions. Sometimes it returns a long list. I do not necessarily triage every question myself. I pass them to the architect who wrote the proposal and ask it to resolve ambiguities or update the document. If the developer still has questions, the exchange repeats. I consider the proposal ready when I understand it and the developer no longer needs important decisions clarified.

This preserves a useful division of attention. The developer explores the code to understand how the agreed change fits. The architect's earlier exploration provides continuity and is responsible for resolving questions about the proposal. A thirty-question handoff is not a signal that the developer should guess. It is evidence that the design context or its explanation may still be incomplete.

The developer does have room to exercise judgment. It may choose where a local file belongs when the repository conventions leave that open. Changes to a dependency, data schema, or API contract are consequential enough that I generally expect them to be visible in the proposal first. The exact boundary depends on the project, but the point is to avoid silently making decisions that alter the system's contracts.

I also want an agent to point out when an instruction appears unsound. If it finds a real problem, it should explain the concern and, where useful, suggest an alternative. For me, this usually indicates that an earlier stage failed to make a constraint visible. We can pause, correct the decision, and

continue. I do not want disagreement to be suppressed simply because the task was phrased as an instruction.

In my current workflow, the reviewer is usually the architect who wrote the proposal. I return to that conversation with the pull-request link, and the architect examines the implementation against the need and the existing project. This keeps the proposal and review connected. Another reviewer could provide a complementary examination, but a second agent's approval does not establish correctness by itself.

Sometimes implementation reveals a small clarification that can be reflected in the conversation and proposal. A substantial change to the design should return to the architect for a revised review. My handling of these changes is not yet fully systematic. The important point is to make consequential changes visible instead of letting the code become the only record of a new decision.

Automated Checks Are the Entry Requirement

My projects have continuous integration with tests and other checks. These checks are prerequisites for review. If they fail, I do not begin my detailed code review.

This threshold keeps the first part of the process explicit. The developer needs to satisfy the project's automated requirements before asking me to assess the broader result.

What those checks establish depends on their coverage. Passing a test suite means that the checks run by that suite passed. It does not prove that every requirement, interaction, or future change has been accounted for.

A successful CI run therefore allows review to proceed. It does not replace the review questions. Does the implementation belong in the places where it was added? Does it introduce avoidable complexity? Does the application behave as intended? Are important assumptions visible?

The distinction also affects how failures are handled. If a check fails, the investigation should explain why. Changing the check can be appropriate when the intended behavior has legitimately changed, but that change needs its own justification. Making the pipeline green without preserving the meaning of its checks would weaken the threshold.

These are general implications of using automated checks as a review prerequisite. The exact commands and coverage should be defined for the project rather than assumed from the existence of a CI pipeline.

The Developer–Reviewer Loop

After the proposal is ready, the developer agent implements the feature in its own worktree and branch and creates a pull request. I ask the review agent to examine that pull request. It gives me findings, which I pass to the developer. The developer makes corrections, and the review follows the updated version.

Useful findings need a basis. They should identify the behavior or structure being questioned, explain why it matters, and make the relevant evidence inspectable. A comment such as "this is not senior code" gives the developer no concrete path to a correction. A finding that names a duplicated source of truth, a missed permission check, or an unnecessary dependency gives the developer something concrete to examine.

The loop should remain tied to the current pull request. A correction can affect neighboring behavior or introduce a new issue. The reviewer therefore needs to inspect the revised implementation, not merely confirm that the previous comment was answered.

The developer and reviewer also have different jobs. The developer changes the code. The reviewer tests whether the change meets the proposal and identifies concerns. I route the findings between them and decide when the result is ready for my own review.

This process reduces the amount of detailed review I have to do on an initial implementation. It does not remove my responsibility for the product or code I accept. The automated checks, the reviewer, and my own judgment each examine different evidence.

A useful round should resolve a known issue, clarify an uncertainty, or provide missing evidence. If the same question keeps returning without a clearer conclusion, the problem may be in the proposal, the source information, or the review criteria. Repeating the loop is valuable only when it moves the work forward.

My Own Review Still Matters

When the reviewer considers the result clean, I begin my own review. I read the code and check the application where the change affects its behavior or appearance.

I look for code that is misplaced, approaches I do not want in the project, and behavior that differs from the feature I intended. In my recent experience, the remaining changes are often small: cleanup, moving files, or adjusting details. That is my observation of the workflow, not a guarantee about every future pull request.

My review has a different reference point from the automated checks. I need to judge the change as part of the product and the repository I am responsible for maintaining.

Preferences are easier to apply consistently when their reasons are explicit. If I reject a file placement, which responsibility or dependency boundary does it violate? If I reject an abstraction, what unnecessary work does it introduce for the next change? If I reject behavior, which requirement does it fail to meet?

Those explanations can improve the current correction and inform the project's documentation. Without them, agents may encounter the same preference again as another surprise during review.

Two Different Reasons to Reject a Version

In my experience, an unsatisfactory implementation often leads back to one of two areas: the initial document or the quality of the implementation.

If the proposal was inadequate, the developer may have implemented something that should never have been proposed in that form. Correcting individual lines of code will not resolve an unclear need or an unsuitable design. We need to revisit the proposal.

In other cases, the intended feature remains reasonable, but the implementation becomes too complex or sacrifices quality. Then the work concerns how the feature has been built. Sometimes I choose to restart the implementation.

What the review reveals	What needs to be reconsidered
The proposed behavior does not meet the need	The product requirement and scope
The proposal leaves an important integration decision unresolved	The technical proposal and its assumptions
The agreed design is reasonable, but the code is unnecessarily complex	The implementation approach

What the review reveals	What needs to be reconsidered
A concrete defect has a clear, limited correction	The affected code and appropriate verification

The table is an editorial decision aid derived from these distinctions. It is not a formal rule I already apply mechanically.

The important step is to identify where the problem originates. Otherwise, a team can spend several rounds repairing an implementation whose governing decision remains wrong.

Complexity Can Become a Reason to Restart

I sometimes start again when an implementation becomes too complex and begins to sacrifice quality. The decision is based on examining the work rather than on a precise numerical rule.

A restart is not automatically the better choice. A difficult feature may require real complexity, and a new branch can reproduce the same mistakes if the underlying uncertainty remains unresolved.

Before restarting, it helps to state what will change. Are we narrowing the scope? Replacing an unsuitable abstraction? Returning to a clearer integration? Revising the proposal? The new attempt needs an improved basis.

The work already completed may contain useful knowledge even if the code is discarded. Findings about constraints, rejected approaches, and successful experiments can be retained in the proposal or supporting documentation.

That makes a restart a decision informed by the previous attempt. It reduces the risk of treating a fresh implementation as a substitute for understanding why the first one deteriorated.

A Week of Active Work Is a Signal

I do not have a strict rule for restarting a design. One practical warning sign is a branch that has occupied us for more than a week.

A branch waiting for feedback is different from a branch receiving a week of continuous work without a satisfactory outcome. The elapsed time alone does not tell us what is wrong.

In the latter situation, I want to understand why progress is slow. The scope may be too broad. The proposal may be incomplete. An integration constraint may still be unresolved. Corrections may be adding complexity faster than they remove defects.

These are possibilities to investigate, not causes established by the age of the branch. Large tasks can legitimately take longer. Smaller tasks can also become stuck much sooner.

The value of a time-based signal is that it prompts a discussion about progress. What have we resolved? What still prevents acceptance? What would make the next round materially different from the previous one?

An answer should lead to a concrete choice: continue with a defined correction, investigate an uncertainty, narrow the task, or revise the approach.

Preparation Can Make a Small Feature Move Quickly

A recent admin-dashboard feature was completed in less than twenty-four hours. I attribute that experience to taking the time to define the feature clearly and keep its implementation appropriate to the need.

This is a single case from my own work. It does not establish a general delivery time, nor does it isolate preparation as the only cause. Scope, the existing foundation, and the particular implementation all affect the result.

It does illustrate the outcome I want from the process: the feature moves through proposal, implementation, corrections, and review without becoming a prolonged foundational investigation.

Reusable foundations support that process by reducing recurring integration work. The feature still requires decisions and verification. A clearer starting point allows more of that effort to address the feature itself.

Make Each Iteration Answer a Question

The cost of an early version includes the attention required to understand it and the work required to correct it. Generating more versions is useful when each one improves our understanding or the implementation.

Before requesting another round, identify the question it should answer. Is a behavior incorrect? Is an interface unsuitable? Is the evidence incomplete? Has the requirement changed?

Then make the expected result of the round explicit enough to review. A concrete correction provides a stronger reference than asking the developer to make everything cleaner.

This does not require predicting every issue in advance. It requires keeping the relationship between findings, decisions, changes, and verification visible as the work evolves.

Over time, recurring findings can also improve the foundation and its guidance. A convention repeatedly clarified in review may need documentation. A repeated integration correction may belong in a reusable component. A recurring blind spot may call for an additional check.

Early Versions Should Improve the Next Attempt

An early version gives us evidence about the proposal and its implementation. Review turns that evidence into decisions about what to keep, change, investigate, or restart.

The process becomes more valuable when its lessons are retained. Otherwise, the next project may pay for the same discovery again.

Preparing the proposal, applying automated prerequisites, examining the implementation through a separate review, and making the final judgment are the main stages of my current practice. They continue to evolve as I learn what needs to be made explicit.

The next chapter expands the problem further: what happens when the product includes several applications and interfaces whose changes must remain coherent?

Chapter 4: When the Product Outgrows a Single Application

First draft of the playbook "From AI-Generated Code to Production Software."

The Product Is More Than Its First Interface

The default scenario in this playbook is a SaaS or web application. The first version may need only a public website and an authenticated dashboard. It does not need a mobile application, a CLI, or an SDK simply because those interfaces might be useful one day.

My SaaS foundation already treats some of these surfaces as separate applications. The public website is for visitors and does not require an account. The dashboard is the authenticated product interface. Documentation has its own application. The applications sit beside shared packages in the monorepo, and each has a distinct responsibility.

That distinction matters because a product's capabilities and the interface through which someone uses them are different things. A dashboard can let an organization owner invite a colleague. A CLI might later allow an administrator to perform a related operation from a terminal. The interfaces differ, but both should use the product's established rules for identity, permissions, invitations, and data.

Thinking this way helps when another interface becomes a real need. The question is not only how to build another screen. It is whether the product has made its underlying capabilities available to another client without duplicating or bypassing them.

Applications Are Clients of Shared Capabilities

A web application, a desktop application, a mobile application, and a CLI have different execution environments and different user interactions. They may still be clients of the same product.

In my architecture, applications consume shared packages and server interfaces. The application owns its presentation and environment-specific behavior. The packages and server expose capabilities that more than one application may need.

For connected applications, the client calls the server through the API. The dashboard uses the RPC client. A desktop or mobile application would use the appropriate client integration for its environment. None of these applications connects directly to the product database.

The distinction is useful in a concrete flow. Suppose an organization owner invites a colleague from the dashboard. The dashboard gathers the input, then asks the server to perform the operation. The server identifies the caller, checks the relevant permission, applies the invitation rules, writes through the data layer, and initiates email through the mail capability. If a CLI later supports member administration, it can call the appropriate server operation rather than implement its own database access and authorization rules.

That gives each layer a job. The client presents and collects information. The API exposes an operation. Server-side code owns product rules and data access. A capability such as email handles

the external side effect. This is an example of my chosen architecture, not a rule that every software product must follow.

Authentication also has a client and server side. The dashboard, CLI, desktop app, or mobile app may need a different sign-in or credential flow, but each flow should connect to the intended product identity and rely on server-side access checks. Sharing an identity system does not require pretending that a browser, a terminal, and a mobile device have identical security constraints.

The server boundary should be explicit enough that a developer can tell which code can safely run in a client and which must remain on the server. A directory name alone may not make that clear, especially in frameworks that execute code in both environments. The integration needs to communicate where credentials, database connections, and authorization decisions belong.

The applications in my current SaaS foundation make the distinction tangible:

Application surface	Its own responsibility	Shared capability it may consume
Public website	Public-facing experience without an account	Public information and any explicitly supported public endpoint
Dashboard	Authenticated product experience	Authentication client and RPC API
Documentation site	Product and technical documentation	Published documentation content
Future CLI or desktop client	Environment-specific interaction and sign-in	Server operations exposed through the product API

This table describes the roles I have in mind, not a guarantee that every future application is already implemented. It also avoids treating every surface as the same kind of application. The public website I described is purely front end. The dashboard is the client that needs authenticated product operations. A documentation app has a publishing and reading role. If the product later adds a CLI, that client will need its own interaction and authentication flow.

A shared package can provide code or a client that belongs in more than one application. It should not erase the distinction between browser and server responsibilities. The API remains the boundary for operations that read or change product data. That way the dashboard does not need a database client merely because the repository is a monorepo, and a later client can use the same operation without copying its business rules.

The API is more than a convenient transport. It is the place where the product can provide a stable operation while its server implementation changes. If the client depends on the database's schema, a database refactor can become a client change. If it depends on a product operation, the server can change its internal storage while preserving that operation's behavior. That benefit depends on keeping the contract intentional and verifying consumers when it changes.

Reuse Does Not Mean There Are No Dependencies

I describe the common pieces as independent bricks that work together. By independent, I mean that their responsibilities are separated and their interactions are explicit. I do not mean that each block operates with no dependencies.

Authentication may depend on a database. An API operation may use authentication and business capabilities. A web application depends on the API contract. An admin dashboard may depend on several packages. These relationships are normal. The architectural question is whether they are visible and controlled.

A package can make a responsibility reusable inside the repository. A server API can make an operation available across different execution environments. They solve related but distinct problems. Sharing source code does not mean every client should execute that code directly. Exposing an API does not automatically make its contract stable or easy to consume.

A useful boundary limits what a consumer needs to know. If the dashboard calls an operation through an agreed contract, the database implementation can change without forcing the dashboard to understand the schema. If several packages share a capability, their dependency direction should be understandable. If a block relies on another block, that relationship should be explicit.

A monorepo helps me keep applications, packages, and development conventions together. It does not create modularity by itself. Applications in separate repositories can still share a product through APIs and published packages. A single monorepo can also contain tightly coupled code that is difficult to extend.

The important test is whether an application can use the capability through the interface it is meant to consume. Can the public website remain public? Can the dashboard authenticate and call server operations? Could a new client use the same product rules without connecting directly to the database? Can a package be reused without importing responsibilities that belong in another runtime?

An Additional Application Tests the Boundaries

Adding another application can reveal what the existing product has actually made reusable.

If an operation exists only inside a web-specific component, another client may require a new interface or a restructuring of the operation. If user identity is assumed to come from a browser session, a CLI may need a different authentication flow. If the client reads the database directly, a second environment may need its own database connection and may duplicate permission checks.

Those discoveries do not prove that the first design was wrong. It may have been appropriate for the original product. They show that the new requirement changes which boundaries matter.

The new interface should have its own needs considered. A CLI needs terminal commands and a way to authenticate. A mobile application needs mobile interaction patterns and credential handling. A desktop application has packaging and distribution work. An SDK needs a public contract and a way to communicate compatibility. An MCP server needs to expose selected capabilities in a form agents can use. The shared capabilities remain connected to the product, but each client has its own integration requirements.

The research on modular systems adds a useful caution. An empirical study of 1,310 OpenStack projects found that cross-component changes were not exceptional. About 10 percent of the studied changes depended on changes in another component. The study describes one large open-source system, so those numbers should not be treated as a forecast for a SaaS template. The broader point is that component boundaries make responsibilities clearer, but do not eliminate changes that cross those boundaries. (Arabat and Sayagh, "An empirical study on cross-component dependent changes: A case study on the components of OpenStack," 2024)

That is why an impact proposal still matters when a product is modular. If a feature touches a database schema, an API contract, a shared package, and two clients, those effects need to be visible. The work may remain straightforward because the boundaries are clear. It still needs to account for each affected consumer.

When adding a second client, I would inspect at least three paths separately. First, can the client authenticate using a flow supported by its environment? Second, can it express the user's action through an existing server contract, or does the server need a new operation? Third, can it exercise that operation without importing server-only credentials or database code?

A useful completion check follows the actual client behavior. The new application should build and run in its target environment. Its sign-in flow should establish the identity expected by the server. A user action should reach the intended API operation. The server should enforce the permission and data rules. If the same capability is already used by the dashboard, a change to its contract should be reviewed with both clients in mind.

Those checks are not all interchangeable. A package test can show that a client library behaves as expected. It does not establish that a user can sign in on a mobile device. A server test can verify permission handling. It does not prove that a CLI sends the right credential. The evidence should correspond to the boundary being added.

Prepare What Is Repeated, Extend What Is Needed

A foundation should prepare the recurring connections that a supported application requires. My SaaS template provides the common structure and integrations that I have already assembled. In the current workflow, the applications and packages are separated, and the basic relationships around authentication and the API are part of the foundation.

The goal of the block system is to make supported additions automatic. A project should not need a developer to manually connect every package after adding an application or capability. The user or agent should invoke the supported operation, provide any project-specific input it asks for, and receive the prepared integration.

The distinction between what exists now and what the system should eventually provide matters. My templates are still evolving. I have not claimed that every future interface or block is already available. The aim is to turn more of the integration work into prepared, repeatable operations as the product grows.

This also helps explain why the system begins from an application type rather than from every capability imaginable. A SaaS web product, a standalone CLI package, and a desktop client have different foundations. The template should establish the relevant application structure. The product can then receive the additional blocks it needs.

A standalone tool published to a package registry is a different case from a CLI that calls a product's server. The standalone tool may have no shared account system or server database. Its design should follow the package's own runtime and distribution needs. This book focuses primarily on connected product applications because they represent the common SaaS case I am preparing for.

Do Not Build Interfaces Before There Is a Need

A product does not benefit from every possible application just because its architecture can support them. Each interface has its own lifecycle, deployment or distribution, testing needs, and expectations. Adding it creates work to maintain even if the initial connection is easy.

Before adding a surface, identify what user or product need it serves. Then determine which existing capabilities it must consume, which client-specific behaviors it requires, and whether current contracts support those interactions. If the API already exposes the needed operation, the new client may be mostly an interface and authentication problem. If it does not, the proposal should identify the server capability that must be added.

This examination can reveal whether the existing boundaries are sufficient. It can also expose a missing block or integration that could be useful to more than one project. Reuse should follow demonstrated recurring needs, not the desire to make a template appear complete.

The practical exercise is to choose one plausible next interface and trace a single user action through it. Identify the client behavior, authentication flow, server operation, data access, and any side effects. Mark which parts are already prepared, which need a product decision, and which need technical research.

That trace tells you more than a list of future applications. It shows whether the product's capabilities are actually reusable and what the next addition would require.

Chapter 5: Composing the Foundations Your Product Needs

First draft of the playbook "From AI-Generated Code to Production Software."

Keep the Starting Point Fast Without Making Every Capability Mandatory

My SaaS template changed what it felt like to start a project. I could clone a foundation, make the project identity coherent, provide the necessary environment values, deploy it, and begin product work. I did not have to repeat the same research and integration work at the beginning of every application.

The next problem was that not every product needs the same capabilities. Authentication, a database, an API, cache, object storage, email, background tasks, and payments are common, but they are not universal requirements for every project. Product-specific capabilities vary too. One project may need an admin dashboard, another a blog, another a CLI or SDK.

The design has to preserve two benefits at once: a fast starting point and the ability to select only what a product needs. A single template containing everything makes optional capabilities part of every project. Creating a separate template for every combination creates a different problem. Each variant needs maintenance, and the number of possible combinations grows quickly as more capabilities are added.

I arrived at a system of blocks. A product begins with the smallest foundation for its application type, then adds prepared capabilities when its requirements call for them.

A Meta-Template Establishes the Application

I use "meta-template" for the minimal foundation that represents an application type. It might establish a web application, a desktop application, or another supported starting point.

The meta-template should be deliberately boring. Its purpose is to give the project the structure required to exist as that kind of application, with the conventions and checks that make it usable. It should not silently impose every feature that a SaaS product might later need.

The minimum foundation differs by application type. A web application needs its own entry point, build and development setup, and a place for its client code. A standalone CLI package has different runtime and publishing needs. A desktop application needs its packaging boundary. These are not interchangeable starters.

Once the application base exists, the project can receive additional capabilities. That keeps the foundation small and makes the product's chosen shape visible. Someone inspecting the project can ask why each capability is present and what responsibility it provides.

A meta-template is not intended to be the only reusable unit. It is the place from which composition starts.

Blocks Represent Capabilities and Their Relationships

A block is a prepared addition to a project. Some blocks provide technical capabilities such as authentication, a database, an API, cache, storage, email, background processing, or payments. Other blocks provide product capabilities such as a blog, an admin dashboard, a CLI, an SDK, or documentation.

The type of a block should make its role understandable. A database block must satisfy the expectations of a database capability. An authentication block should provide the operations that the rest of the system expects from authentication. A cache block should meet the cache contract used by its consumers. The project should be able to distinguish different implementations of the same type without pretending they are identical in their configuration or behavior.

This is where alternatives can fit. Authentication may be provided by several libraries or services. Those implementations can be separate blocks of the authentication type. A common contract allows a consumer to understand the expected capability, while each block declares the specific provider, dependencies, supported configuration, and constraints it brings.

Blocks also have dependencies on other blocks. An authentication implementation may require a database. A feature block may require authentication and an API. A cache may only be appropriate for a workload that needs it, rather than being a mandatory consequence of having a database.

The dependency belongs to the selected implementation, not necessarily to every block of that general type. The system needs to distinguish requirements from recommendations. It should not add cache because one database option happens to use it, unless the product has explicitly chosen that integration.

Some blocks may help initialize another block. That is a possible relationship in the design. The important product-level behavior is that dependencies and supported setup are represented in a way the system can act on. The internal mechanism can evolve without changing the user's mental model: blocks provide typed capabilities, declare relationships, and cooperate within a project.

The shadcn Registry Is a Useful Analogy, With Limits

The shadcn registry demonstrates a concrete approach to distributing optional resources into a project. Its registry items declare a type, files, package dependencies, dependencies on other registry items, and supported configuration. Its CLI can resolve registry dependencies and add the described resources to a project. Namespaces allow resources to come from different registries. (shadcn/ui, Registry overview, Registry item schema, Namespaced registries)

The analogy is useful because it makes the experience tangible: a project can discover a resource, inspect its description and dependencies, then add it through a command. A user does not need to copy files and manually resolve every registry dependency.

There are important differences. The shadcn registry primarily distributes source files and configuration into an existing project. That model does not by itself guarantee that a server integration works, that data migrations are safe, that external infrastructure is provisioned, or that an authentication provider is correctly connected to every application. Those relationships require additional contracts and verification.

My block system aims at a broader integration problem. It should not only place implementation files. It should connect the selected capability to the project's applications and server packages according to the supported contracts. shadcn is an example of the compositional interaction pattern, not proof that the entire system design is already solved.

One Command Should Perform the Supported Addition

The intended behavior is straightforward: the user or agent runs a command to add a capability, and the system performs the supported installation and integration.

If the selected block requires other blocks, the operation should discover and add those dependencies. If the addition needs a provider choice, a key, or another project-specific answer, it should ask for that input during the addition. The user should not have to run a series of unrelated commands and then repair the connections manually.

The command is only simple from the user's perspective. Behind it, the block needs a defined type, support conditions, dependencies, integration behavior, and appropriate validation. That preparation is substantial. Its value comes from performing the same work once and applying it consistently to the projects that need it.

For a selected capability, the user-facing operation should hold together as one coherent addition:

1. The user or agent identifies the block and sees what capability it provides.
2. The system resolves the block's required dependencies and checks that the project type is supported.
3. It asks for any provider choice, key, or project-specific answer that is actually required.
4. It installs the prepared implementation and connects it through the supported interfaces.
5. It reports what was added and runs or identifies the relevant verification.

This is a product-level description, not a claim about the command's internal algorithm. The user should not need to understand the resolver or package graph to add a supported capability. The implementation still needs to handle those details reliably.

For example, a new SaaS might begin with its web application foundation and later need authentication, member invitations, and email. Authentication may require a database block. The invitation feature may rely on the API and the mail capability. The user should be able to request the relevant addition, answer the provider questions that appear at that point, and receive the connected implementation. The agent can then implement the product-specific invitation behavior using the prepared interfaces.

The feature itself is not automatically complete just because its foundation exists. Product rules still need to be chosen, such as who may invite a member and which role is assigned. The block system prepares the technical capabilities the feature can use. The product workflow is still designed and reviewed as described in the preceding chapters.

The operation should also make its result understandable. The user or agent needs to know whether the addition completed, which input is still required, and what was established. An installed integration that still needs real provider credentials is different from an integration that has already communicated with the external service. The command should not imply a stronger result than it has verified.

These are requirements for the system I am building toward, not a claim that every block or combination is currently implemented. The command-based addition is the target experience.

Ask for Inputs at the Point of Addition

Some capabilities require project-specific choices. A database block may offer several supported providers. An authentication block may need a provider choice. Storage may require credentials or an existing account. Those questions should appear when the user or agent adds the relevant block, because that is when the answer becomes necessary.

This is different from asking every question at the start of a project. The meta-template should not ask about payments if the project is not adding payments. A capability that is never selected should not require configuration.

The prompts must also work with agent-driven workflows. If an agent runs the command, the required choices need to be discoverable and the inputs need a supported way to be supplied. The command should be able to ask questions when interaction is available, while also making it possible for an agent workflow to provide explicit answers. The precise interface is an implementation detail. The requirement is that an agent can complete a supported addition without inventing missing values or being left at an unexplained prompt.

The system should not confuse installation inputs with product decisions. A command can ask which supported mail provider to use. It cannot decide whether the product should email users or what the email should promise. That remains part of product conception.

Blocks Should Be Discoverable Before They Are Installed

A marketplace or catalog can help a user or agent find a block suited to a need. The system should make it possible to search or list blocks, read a block's description, inspect its type and dependencies, and understand its supported behavior.

An agent may use that information to propose a block or to add one after the project requirement is clear. This should reduce the need to search the web for every repeated integration. It does not remove the need to evaluate whether the block fits the product.

A description that only names the technology is not enough. The user or agent needs to understand the capability, its required dependencies, the environment it supports, and any provider-specific requirements. Otherwise, the catalog only relocates the research burden.

For repeated implementations of one capability, the catalog should let alternatives be compared on meaningful differences. Two authentication blocks may satisfy the same broad role but have different supported environments, data requirements, or setup. A common type helps organize them. It does not make them interchangeable in every project.

This discovery experience is part of the system's value for agents. If an agent can inspect available blocks and their contracts, it can begin from prepared capabilities rather than reconstructing the integration from general model knowledge.

The agent should not have to guess from a short label alone. It should be able to inspect the block's role, type, required dependencies, supported project environments, and the interfaces it exposes. From that information it can compare candidates and explain why one appears to fit the requested feature.

The final choice still depends on the project. A block may technically satisfy a type while not supporting the required runtime or provider. Another may be a stronger fit but require a dependency that the project does not want. Discovery should narrow the options and make tradeoffs visible, not turn a catalog match into an automatic product decision. Once the selection is clear, the command can perform the supported addition.

A Block Must Work With Its Neighbors

A block can function by itself and still interact badly with another block. The composition needs to be verified at the boundaries it creates.

If a functional block depends on authentication and data access, its intended flow should be checked with those capabilities. If adding an API changes a contract used by several clients, the affected con-

sumers need appropriate verification. If two providers satisfy the same block type, each supported implementation needs evidence for the behavior the type promises.

The contract describes what consumers can rely on. Tests and other checks provide evidence that an implementation meets the contract. End-to-end checks can establish that selected pieces work together for an important user action. These forms of evidence answer different questions. A passing unit test for one package does not by itself prove that the dashboard can call the server or that an email operation behaves correctly.

The composition system should make it possible to verify the installed result, not just the source block in isolation. It may need to account for the project type, existing files, dependency versions, configuration, and previous customizations. Those details influence which additions the system can safely automate.

The design goal is automatic behavior within a declared support boundary. If a requested combination falls outside that boundary, the system should make the limitation understandable instead of leaving the project in a partially integrated state.

The Feature Belongs to the Product, Not the Template

Composition keeps the starting point focused. It does not determine which capabilities a product needs.

Start with the application type and the first behavior the product must support. From that behavior, identify which shared capabilities it requires. Then examine which blocks satisfy those roles and what each one depends on.

For a simple public site, the right foundation might need no user authentication or database. A dashboard that manages organizations may require both, along with an API. A product that uploads files may add storage. A subscription product may add payments. These examples are not a checklist of features every SaaS should contain. They show how different requirements lead to different compositions.

The agent can help explore the catalog and explain how candidate blocks fit the feature. The product owner still decides whether that behavior belongs in the product. Composition should make the technical work of adding it repeatable, not smuggle product choices into the starting point.

The block system is the direction I am taking because it preserves the speed I gained from templates while allowing each project to select the capabilities it actually needs. The initial template establishes the application. Blocks add the prepared capabilities and integrations. Product-specific development continues on top of that foundation.

Chapter 6: Designing How the Pieces Work Together

A system of reusable blocks depends on more than finding files and copying them into a repository. Each block needs to make a promise that its neighbors can understand.

In DeesseJS, a block type describes a kind of capability, such as a database, authentication, storage, email, or an API. Blocks of the same type may use different providers. Both should satisfy the expectations that the rest of the system relies on when it consumes a database capability.

The exact mechanism used to express these relationships is an engineering question. This chapter focuses on the design requirement behind it: useful composition needs explicit boundaries, predictable dependencies, and a way to check whether a block can participate.

Composition Requires Shared Expectations

Consider a product that starts with a minimal SaaS application and later adds authentication. Authentication may need a database for users and sessions. The dashboard needs a way to identify the current user. The server needs to validate requests and enforce access. The product may also need email for verification or recovery.

Each capability can be implemented separately, but the product only works when their expectations line up. The authentication block must know what data persistence it can rely on. The API must know how identity is made available. The dashboard must know what client interface exists. Email must be called in the way the selected provider supports.

If one block silently assumes another block's internal structure, the overall system becomes fragile. A provider change, file movement, or upgrade can then force edits in unrelated places.

The Software Engineering Institute describes integrability as the cost and risk of combining components. Its guidance emphasizes that dependencies are not only code references. Components can also be coupled through shared protocols, file formats, runtime resources, or assumptions about behavior. A visible module boundary therefore does not prove that two components are independent. SEI, Integrability

For this system, a practical implication follows: define what consumers may rely on, and keep provider-specific internals behind that boundary.

Make Future Feature Decisions Smaller

The purpose of a block system is not to create an abstraction for every library. It is to make recurring composition more predictable.

Without a shared expectation, each feature starts by asking how to integrate a capability. The developer agent may need to rediscover which package owns persistence, how the API reaches it, what imports are allowed, and where client-side code belongs.

With an agreed boundary, the question becomes smaller: does the chosen block provide the operation the feature needs, and how should the consumer call it?

This does not eliminate technology choices. A product may need a provider with a specific deployment model, performance profile, or licensing condition. Nor does it guarantee that any combination is valid. The goal is to make incompatibility visible before implementation depends on it.

A contract is especially valuable when a capability has multiple implementations. If the product supports only one provider, the contract can still make expectations clear, but it may not justify a complex generic abstraction. The design should match the number of real consumers and alternatives, not an imagined future marketplace.

A Type Identifies the Capability

The type tells the system and the reader what role a block claims to provide. Authentication says more than a package name tied to one provider. It lets another block express a dependency on the capability rather than on a specific implementation.

An application could require an authentication block and a database block. It should not need to know every implementation detail of every possible authentication provider before it can express that need.

The type should not be mistaken for proof of compatibility. Two blocks can both identify as authentication and still differ in supported flows, user models, session behavior, or runtime constraints. Type names are useful labels. The contract and compatibility rules carry the more important meaning.

This is also why a catalog should describe provider-specific conditions. A block's metadata can tell the agent what the block does, what it needs, and what it provides. A type alone is too small a description for a meaningful selection.

A Contract Describes What Others May Rely On

A contract identifies the stable expectations a consumer can use without depending on a provider's internals. At a high level, an authentication contract might state that the project can obtain the current authenticated identity in a server request and can protect a route using the project's agreed authorization mechanism.

The contract should leave implementation choices open when consumers do not depend on them. If all consumers need only an identity identifier and a role check, they should not rely on the internal session table layout of one provider.

Contracts can be expressed in different ways depending on the boundary: a TypeScript interface, a documented API schema, a command convention, a set of files or exports, or tests that exercise the interaction. No single expression fits every block. The important property is that consumers and implementations share an observable expectation.

Pact's guidance on consumer-provider contract testing illustrates a useful restraint: tests should capture behavior the consumer depends on, not every detail the provider happens to expose. Expectations that are stricter than consumer needs can make providers brittle to evolve. Pact, Consumer testing The same principle applies to reusable blocks: specify enough to make a combination reliable, but avoid freezing incidental details.

A useful contract question is: if a provider changed this detail, would a consumer that follows the contract need to change? If the answer is no, the detail may not belong in the shared promise.

Dependencies Make Relationships Explicit

Dependencies describe what a block needs and what it makes available to other parts of the project. For example:

- An authentication implementation may require a database capability.
- An API block may require server runtime support and may consume authentication context.
- A dashboard may consume the API client and authentication client.
- A storage capability may require credentials supplied for a specific provider.

Dependencies should be directional. A dashboard can depend on an API client, while the server API owns the logic that accesses the database. The dashboard should not gain database access merely because it needs to display data.

In my current SaaS architecture, I use the same boundary: applications are clients of shared packages; the applications do not access the database directly. They call the API. Authentication also exposes a client/server interface. Email and similar services are generally requested through server-side capabilities.

The direction of dependencies matters because it determines how far a change can travel. If all applications depend on the database schema, changing the schema can touch every application. If the server API contains the database interaction and exposes a stable operation, clients are insulated from some internal changes.

The SEI report cautions that service boundaries alone do not remove hidden semantic or runtime coupling. Two components can communicate through an API yet still rely on undocumented behavior. A contract needs to cover the assumptions that matter to the consumer, while verification should examine the actual integration where risk warrants it.

Some Blocks Can Help Initialize Others

Some blocks need information or structure from another block before they can be set up. Authentication might need a database schema or database connection capability. A dashboard block might need the API client to exist before it can configure typed operations.

A composition system can model these relationships so that adding one capability identifies its prerequisites and arranges the supported setup. The user should see any required choices or secrets at the point they matter, such as provider credentials during the addition flow.

This is a desired behavior of the block system, not a claim that every capability already supports it. The system should be explicit about which relationships it understands. If a dependency is missing, incompatible, or requires a manual decision, that condition should be surfaced before files are changed.

The order of setup may also matter. A database capability may need to exist before an authentication capability can create or validate its schema. But a visible sequence is not the whole design. The resulting packages still need to communicate across the contracts they share.

Conventions Give the Project a Predictable Structure

A block can satisfy a functional contract and still make the repository harder to work in if it places files unpredictably or ignores local conventions.

The project therefore needs conventions for ownership and organization: where server logic belongs, where a client package lives, where app-specific screens go, how generated files are named,

and where provider configuration is kept. These conventions help both people and agents locate the extension point for the next feature.

Conventions should be consistent enough to reduce guessing, but they should not create unnecessary constraints. If an external package or existing project has its own structure, a block may need to adapt to it rather than rewrite it. The aim is a coherent project, not the appearance of uniformity at every file.

A useful design review asks both questions: does the integration work, and can the next contributor predict where to find or extend it?

Existing Code Can Coexist with the System

A block system should not assume that every useful component originated inside the system. A project may already use a library, a generated component, or a convention that is not part of the block catalog.

The composition system can coexist with that code if it preserves clear ownership. Existing components may be treated as external dependencies, adapted behind a contract, or left untouched when they do not conflict with the new capability.

The difficult case is a mismatch. Perhaps an existing package uses a different API shape, or two components assume different ways to provide authentication context. Then the right response may be an adapter, a migration, or a deliberate decision to keep the integration separate. Quietly copying both solutions into the same project without clarifying ownership creates hidden coupling.

No system can make every combination work automatically. A trustworthy catalog needs to state its supported combinations and assumptions rather than imply unlimited compatibility.

Shared Expectations Reduce Repeated Integration Decisions

When a feature is planned, the architect should identify the affected capabilities and applications: which data tables or schemas change, which API operations are added, which clients consume them, and what configuration or external providers are involved.

That description is more useful than merely listing files to edit. Files can move. The system relationship is what the reviewer needs to evaluate: who owns the behavior, what contract changes, which clients are affected, and how compatibility will be established.

During implementation, the developer should explore the repository and ask questions that expose mismatches between the proposal and the code. If adding a capability reveals that the shared expectation is incomplete, the question returns to the architect before the implementation continues on a false assumption.

This keeps the agent free to notice problems while preserving a clear decision owner. It also limits the chance that the developer solves an architectural uncertainty by making an undocumented local choice.

Internal Mechanisms Can Evolve Within Defined Boundaries

The machinery that discovers blocks, resolves dependencies, writes files, checks compatibility, and prompts for configuration may change over time. It is not the central promise of the user-facing system.

The stable idea is simpler: a block claims a type, describes its requirements and contributions, and participates in a project through explicit contracts. Some blocks can initialize or enable others. A command should perform supported additions automatically and request only the inputs required for that choice.

The internal engine can evolve as experience reveals better ways to implement this behavior. The contracts and user-visible outcome should remain understandable enough that a project is not tied to unexplained internals.

Evaluate the Relationships Before the Engine Details

When considering a block or integration, ask:

1. What capability does this block provide?
2. What stable behavior can consumers rely on?
3. What other capabilities does it require?
4. What does it expose to the rest of the project?
5. Which provider-specific assumptions must remain visible?
6. What will fail if a neighbor changes its implementation?
7. How will the supported interaction be verified?

These questions focus on the relationship between pieces. The precise packaging, metadata format, and installation algorithm can be chosen and revised later.

Composition is successful when the project gains a capability without forcing every consumer to understand its internals. It is not successful merely because files were added by one command.

Chapter 7: Giving Agents the Context Their Role Needs

An agent does not need every document you have. It needs the information that lets it make the decisions assigned to its role, plus a reliable way to find project-specific facts it must not guess.

That distinction matters in a workflow with separate people or agents for architecture, implementation, and review. A product proposal may include the reasoning behind alternatives, open questions, and consequences. A developer implementing an approved change often needs the decision, its constraints, affected packages, and the relevant code conventions. Sharing every historical discussion with every role can bury the actionable information under material that no longer matters.

Context Follows Responsibility

Start by asking what the agent is responsible for in this task.

An architect investigating a new feature needs broad enough access to understand the system, explore current documentation, compare alternatives, and record important tradeoffs. The architect may need to edit product or architecture proposals.

A developer agent receiving an approved task needs to understand the intended behavior, the project conventions that affect implementation, and the code it will change. It may need to inspect the relevant package documentation and source files. It does not necessarily need the history of every alternative considered during product conception.

A reviewer needs the change, the acceptance criteria, the expected behavior, and a clear way to inspect evidence. If the reviewer is also the architect, it may need access to the proposal to compare intent with implementation. These responsibilities can be held by one agent across different stages, but the context should still fit the stage.

A role is not a model ranking. It is a description of the work and decisions assigned to the agent.

The Architect's Workspace Is a Different Kind of Workspace

In my workflow, RFCs, ADRs, product exploration, and design discussions belong in a separate internal workspace rather than being committed into the software repository by default. I want to revise these documents as the discussion changes without turning every exploratory edit into repository history.

The architect is responsible for reading and updating that material. The developer is generally assigned the approved result, not every step of the prior debate. This reflects a division of responsibility: product and architecture decisions are developed before implementation, while the developer uses the resulting context to produce the requested change.

This does not make the decision inaccessible or secret. It means the reader sees it in the workspace designed for product and architecture work, while the repository contains the implementation and durable contributor guidance relevant to that code.

There are limits. A decision may need to be represented in code, tests, an API schema, or a short repository convention when future contributors need it to work safely. Keeping all rationale external does not help if the implementation itself gives no clue about its supported behavior. The question is not whether an explanation is useful. It is where it should live and which role needs it.

Give the Developer the Result of the Decision

For a feature, the approved proposal should present what it affects and what it allows the product to do. It should identify impacted applications and packages, relevant tables or schemas, API operations, authentication implications, external services, and important acceptance behavior.

It need not be a line-by-line instruction to edit file A, then file B, then file C. That level of direction can be brittle and can prevent the implementer from discovering the right structure in the current code. The developer should explore the repository and choose local details such as file placement where the architecture leaves them open.

At the same time, some choices should not be left to implementation. In my current process, dependencies and schema changes are generally shown in the proposal for review. The developer may suggest a better approach or identify a flaw, but should explain the reason and return the issue for a decision.

This is a useful division between an outcome-oriented description and a coding recipe. The first defines the affected system and the expected behavior. The second dictates implementation steps. Use the second only when a constraint truly requires it.

Keep Repository Guidance Focused on Development

The repository still needs concise, durable guidance that helps a developer or agent work safely. In my projects, a root Markdown entry point describes the project and links to more focused files for packages, applications, or internal development rules.

A useful root document answers immediate orientation questions:

- What is this repository and what does it contain?
- Which application and package boundaries matter?
- Where can the agent find the conventions for the area it will change?
- Which commands run the relevant checks?
- Which technologies or patterns should it verify against current documentation?
- Which dependencies must not be bypassed, such as a client importing a database driver?

The root file acts as a map, not an encyclopedia. A package or application can have its own entry point with specific conventions. A deeper documentation file can be linked when the task requires it. This makes context navigable without forcing every rule into one huge prompt.

The structure should reflect real work. A small package may need only a few lines. A complex application may need a focused guide. Do not create documents merely to imitate a documentation scheme.

A Small Entry Point Can Provide a Map

A simple pointer system can be enough: one Markdown file explains the repository and links to other relevant files. Frameworks and coding-agent tools vary in how they load instructions, and agents do not always read every relevant file automatically.

The safe assumption is that an agent may miss context unless the task and the tool's actual behavior make it available. If the tool loads repository instruction files automatically, use the documented mechanism. If it does not, include the important entry point in the task. Then link to the more specific guidance the agent should inspect.

The amount of text is not the only measure of context quality. A small map that points to current, relevant material can be better than a large file with stale details. Conversely, a brief instruction that omits a critical constraint still leaves the agent to guess.

For example, a developer tasked with adding an API operation might receive the feature proposal plus a pointer to the API package guide. That guide could identify the current RPC framework, route conventions, validation pattern, relevant command, and test location. The agent can inspect the actual package and current upstream documentation rather than relying only on a copied tutorial.

Document Lessons, Then Enforce What Can Be Enforced

I do not yet have a reliable process for capturing every recurring agent mistake. This is an open weakness in my own workflow, not a solved capability.

The useful direction is to record a lesson when a problem reveals a durable constraint, then ask whether it can be enforced deterministically. A rule such as "do not import the database package from a browser client" may be better represented by a package boundary, lint rule, import restriction, or build check than by another paragraph that an agent may overlook.

Some lessons cannot be reduced to a static rule. A reviewer may need to assess whether a proposal changes a public API compatibly, whether a schema operation matches a product requirement, or whether an abstraction is justified. In those cases, provide the reviewer with the rule, the affected code, and enough evidence to make the judgment.

Tests and static checks do not replace documentation. Documentation explains intent and constraints; automated checks catch a defined class of violations. When they overlap, prefer a deterministic guard for behavior that can be detected reliably and keep prose focused on reasoning a tool cannot infer.

Permissions Should Reflect the Roles

Context is not only what a role can read. It is also what it is permitted to change.

In the current workflow, agents work in separate environments or worktrees, use their own branches, and propose changes through pull requests. The developer can change assigned code but should not be able to accidentally destroy unrelated work or directly overwrite the protected primary branch. CI checks are prerequisites for review. A reviewer can examine and request changes. The owner makes the final decision.

Internal product and architecture documents can be external to the repository and accessible to the architect role. If developers do not need to edit them, access can reflect that division. The exact access controls depend on the systems being used, but the intention is straightforward: grant enough context and ability to perform the task, while limiting unnecessary write access.

Isolation protects both the project and the quality of the process. A helpful agent can still misunderstand an instruction, run a destructive command, or affect the wrong branch. Separate environments, restricted credentials, protected branches, and review checkpoints give the work a boundary.

Keep the Current Decision Available

Architecture and product documents often change during discussion. A proposal is useful only if the version used for implementation reflects the agreed decision. When a developer agent asks questions, my current process is to pass those questions to the architect, which decides what matters and updates the proposal where needed. I usually forward the questions without filtering them myself.

That process helps expose gaps before implementation, but it also has limits. Questions that look repetitive may signal a missing instruction; questions that ask the developer to choose a major dependency or schema change may signal that the proposal has not yet settled the issue. The architect should separate questions that need a decision from details the developer can resolve through repository exploration.

A developer may discover new information after implementation starts. That should not be silently absorbed as a local assumption if it changes the scope, dependency, schema, or product behavior. The change returns to the architect or owner for review. Smaller clarifications may be recorded in the proposal or task context. Larger changes may require another revision before coding resumes.

This keeps the decision current without requiring the developer to carry the full history of how it was reached.

Chapter 8: Verifying Foundations Before Reusing Them

A foundation can look convincing in a repository and still fail in the first project that depends on it. Verification is how we distinguish a useful starting point from a collection of promising source files.

In my own work, a template is usable when it satisfies the needs of my personal projects. I inspect the code and rely on CI, but I do not yet have one formal score or an exhaustive certification process for every possible use. That distinction matters: a standard for my use is evidence about my needs, not proof that the template supports every team's deployment or every product shape.

Verify the Base and the Changes Separately

There are two related questions:

1. Does the foundation behave as it claims in a clean project?
2. Does a particular feature change preserve the behaviors the project relies on?

The first concerns the reusable starting point. The second concerns the new change. A feature PR can pass its tests while inheriting an untested failure in the template. A template can pass its own CI while a particular application configures it incorrectly.

Keep the evidence connected to the claim. If a template says the dashboard calls the API instead of accessing the database, inspect that boundary and check for violations. If an authentication block says it supports a password reset flow, exercise that flow or clearly state that it is not covered by automated tests. If the promise concerns a fresh clone, test a fresh clone rather than only a developer's already-configured checkout.

Verification does not require a massive certification program at the beginning. It requires making the claim specific enough to check and choosing evidence that actually addresses it.

What I Currently Mean by Usable

My current bar is practical. I use the templates for personal projects, examine the code, and rely on CI to catch failures covered by its checks. I am critical about the foundations I use, but this is still a bounded test: it reflects the projects and workflows I have tried.

A stronger validation plan can be built around concrete claims:

Claim	Evidence to collect
A clean project can start	Clone or generate it in a fresh environment and follow the documented setup
The expected apps build	Run the repository's real build commands
Shared packages follow boundaries	Check package imports, public exports, and forbidden dependencies

Claim	Evidence to collect
Authentication works for the supported flow	Exercise the relevant success and failure paths
The API contract is consumed correctly	Test representative clients and server behavior
The project can be deployed	Perform a deployment using the supported provider and configuration
A block can be added safely	Add it to a clean compatible project, inspect its effects, and run the resulting checks

Each row makes the claim more precise. "Production-ready" is too broad to verify as a single property. A foundation can be sound for a specific application and deployment while lacking a feature another project requires.

A Test Retains Its Role After It Passes

A passing test is not a reason to delete the test. It is evidence that a defined behavior held under the tested conditions at the time the check ran.

The test remains valuable because later changes can reintroduce the same failure. This is particularly important for template code: a fix benefits projects created from the template, but an upgrade or a block addition can reopen the same boundary.

Tests also have limits. A passing test does not prove the absence of every defect, establish a desired product behavior that was never specified, or demonstrate that a deployment works in an environment the test does not exercise. The test's value depends on whether it covers a relevant failure mode.

When a bug appears, capture a reproducible case. Then decide which layer can reliably prevent it from returning: a unit test, an integration test, a static rule, a boundary check, or a documented review question. Keep the protection at that layer and preserve it in CI when it can run repeatedly.

Separate Different Kinds of Evidence

A useful verification process combines evidence rather than treating one green check as proof of everything.

Static checks can catch known classes of structural problems: type mismatches, lint violations, invalid package imports, missing generated files, or forbidden dependency directions. They are especially helpful for conventions that can be encoded deterministically.

Unit tests can check local behavior, such as token expiry calculations or validation of an input. They are fast and useful, but do not establish that authentication, storage, and API integrations work together.

Integration tests exercise boundaries between components. If the dashboard's API client and server route are expected to agree on an RPC contract, tests can check how the request is formed and how the provider responds. Pact's guidance recommends testing the consumer-provider interactions the consumer actually relies on, while avoiding assumptions about irrelevant provider details. Pact, Consumer testing

End-to-end and deployment checks can exercise a workflow across the running application. They are useful for selected critical paths, but usually take more time and involve more configuration than unit or contract checks.

Human review evaluates matters that automated checks may not settle: whether the change matches the proposal, whether the design is unnecessarily complex, whether responsibilities are in the right place, and whether an assumption should return to product or architecture discussion.

These forms of evidence answer different questions. A healthy process uses the least expensive check that reliably covers a risk, then adds broader checks where the consequence or coupling justifies them.

Check the Behavior the Foundation Claims to Provide

A template often contains capabilities that can each seem correct in isolation. The risk lies in their interactions.

For a SaaS foundation, a realistic check might start with a clean project, create or sign into a user, load an authenticated dashboard, request a server operation, and confirm that data is persisted through the API boundary. The exact flow depends on what the foundation claims to support. The point is to test the path that crosses the important integrations, not merely that every package compiles.

At a narrower level, a template can have checks for package boundaries: a browser application should not import the database driver; a server route should validate its request; a generated API client should match the server schema. Some boundaries are best checked through types or lint rules, others through runtime tests.

Contract tests need care. A contract should protect a consumer from breaking changes to behavior it uses. If it asserts incidental response fields or internal details, it can make harmless provider changes look like failures. Pact recommends keeping assertions focused on what the consumer depends on and using flexible matching when exact values do not matter. That tradeoff applies to other contract tests as well.

Supported Changes Still Need Verification

The block system is intended to make supported additions automatic: select or discover a capability, run its addition command, answer any provider questions, and receive the corresponding files and integrations. That does not mean verification can be skipped.

For a supported block combination, the system should be able to show what changed, what dependencies it installed, which apps or packages it touched, and which checks ran. A successful command means the supported transformation completed. It does not, by itself, prove that the product now behaves correctly for every use case.

For an update to an existing block, the compatibility question is more difficult. The block may have local edits, downstream changes, or consumers that rely on its old behavior. The update should expose conflicts and make them reviewable rather than silently overwriting application work.

Copier is one example of a template tool that can compare a generated project with an updated template and surface conflicts for manual resolution. This is evidence that preserving a relationship to an upstream template is technically possible, not evidence that every repository can be updated without conflict. Copier, Updating a project

Turn Repeated Defects into Retained Protection

A useful quality loop is:

1. Describe the defect in a way another person can reproduce.
2. Identify which contract, assumption, or boundary failed.
3. Decide whether the correction belongs in a reusable block, a template, the product, or the developer process.
4. Add a check at the lowest layer that can reliably catch a recurrence.
5. Run the check in the relevant CI workflow.
6. Update concise guidance if the failure depends on a human decision the tools cannot enforce.

For example, if a browser package imports a server-only database client, the first response may be to remove the import. But if nothing prevents it returning, add a boundary rule or package check. If two clients interpret API errors differently, clarify the API contract and add appropriate tests. If an agent repeatedly chooses an outdated API, point its task to the current documentation and make version checks part of the process.

My own capture of lessons is not yet systematic. I want recurring lessons to become durable protections, but I am still working out how to manage them without flooding the repository with volatile notes. A deterministic check belongs close to the code that it protects. Historical product discussions may belong in the external architecture workspace instead.

Feedback Expands the Context We Can Observe

Personal use is an important source of evidence, but it has a blind spot: I know how my own projects are structured and may unconsciously supply missing context. A first-time user may choose a different provider, deployment environment, or feature combination.

Once other developers use a foundation, feedback can reveal paths I did not test. Issue reports, reproducible examples, and CI failures can help distinguish a general defect from an unsupported combination or a configuration mistake. The quality process should make those reports actionable by identifying the template or block version, the selected options, and the failure point.

That future feedback loop should be described honestly. Building a public issue path and collecting external reports is part of making a product usable by others, but the existence of a plan does not mean a mature support process already exists.

Make the Remaining Uncertainty Visible

Before reusing a foundation, record what you verified and what remains unknown. The goal is not to produce a badge claiming that a template is perfect. It is to help you make a scoped decision.

A small evidence note might say: fresh clone and build tested; authentication and API flow exercised locally; deployment tested on a named provider; alternate database providers not evaluated; adding a block to a repository with local edits not yet supported.

This is more useful than a general label because it tells the next developer what the evidence covers. It also makes it easier to expand coverage when a new product needs a previously untested capability.

The foundation earns trust through repeatable evidence and visible limits. Passing CI is one part of that trust. Clear boundaries, realistic integration checks, and a process for retaining lessons complete the picture.

Chapter 9: Starting a Product on an Existing Foundation

The foundation changes the first day of a project. It does not change who has to decide what the product should do.

My current starting process is deliberately direct. I clone the SaaS template, tell the agent that the repository is based on that template, and discuss the project. The agent adapts project identity where needed, including documentation and package metadata. I supply any environment values or deployment inputs that the project requires. Then we move into product work.

When the foundation is ready, I do not expect to spend the first week rebuilding authentication, database access, and API conventions. I expect to start working.

Start with a New SaaS or Web Application

The main example in this playbook is a new SaaS or web application. A typical project may include a public front-end that does not require an account, an authenticated dashboard, shared server packages, and a documentation application.

The project may need a database, authentication, an API, user interface components, email, storage, cache, payments, or background tasks. Some capabilities are common across products; others are specific to this product. A useful starting foundation is not necessarily one enormous template that includes every possible feature. The block approach described earlier is intended to add the capabilities a project actually needs.

Start by identifying the product's first useful behavior and the applications or services involved. The architecture can remain simple while the ownership boundaries are clear: apps are clients of shared capabilities, and database access belongs behind the server API.

The Preparation Changes What Comes First

Without a prepared foundation, a project may begin with tooling decisions, dependency installation, database wiring, authentication, application structure, environment configuration, and deployment. Each step may expose an integration question that delays product work.

With a prepared foundation, that order changes. You start by adapting the project identity and confirming the environment. Then you can discuss the product's behavior against an existing system.

This is not a claim that every task becomes instant or that every integration is already available. A deployment may still need provider setup. A secret must still be supplied securely. A product requirement may reveal that the selected foundation does not support a needed capability. The difference is that recurring work already solved by the foundation does not have to be rediscovered as part of every project.

Explain What the Existing Base Represents

When you hand a cloned repository to an agent, give it the small amount of context that prevents it from treating the foundation as unexplained code.

For example:

- This repository was created from the SaaS foundation.
- Its shared packages provide the existing database, authentication, and API integrations.
- Applications consume those packages through their client interfaces.
- Do not replace an existing integration before explaining why it does not support the product need.
- Adapt project identity and the relevant metadata before feature work.
- Inspect the current package guides and version-specific documentation before adding or changing a technology.

The actual instructions should match the repository. A generic note is not a substitute for the project's conventions. The point is to tell the agent that the existing structure is intentional and where it can inspect the details.

In my workflow, agents can explore the repository to understand the proposal. That exploration is not an invitation to redesign the foundation for every feature. It is how the developer gains the local context needed to implement the approved change.

Begin with the Need

Once the project identity is in place, describe what you want the product to do. Keep product discovery separate from technology assumptions.

A feature request can begin with a user problem and a desired outcome. If the need is unfamiliar or depends on current market behavior, research alternatives and user expectations. The architect can capture the resulting product decisions in an internal proposal and ask follow-up questions.

The aim is to establish behavior, affected users, boundaries, and important cases before implementation. For an invitation feature, that might mean deciding who can invite, what happens when the invite expires, and whether a user can hold more than one active invite. The foundation can make the implementation path easier, but it cannot answer these product questions.

Choose the Capabilities the Product Needs

After clarifying the feature, identify what it touches in the system: applications, shared packages, data schema, API, authentication, storage, notifications, or external providers.

The foundation helps by making the available capabilities familiar. The block system is intended to let an agent search a catalog, inspect block descriptions and dependencies, and propose the blocks that fit. Supported additions should then be performed through a command, with any required provider choices or secrets requested at that point.

This describes the intended capability and the user's design for the system. It should not be read as a guarantee that every block or combination is currently available. For a current project, check the actual catalog and product documentation. If the required capability is not there, treat it as an explicit gap rather than silently substituting an unrelated package.

Resolve Project-Specific Inputs

The foundation can prepare integrations, but projects still differ in their credentials, provider accounts, domains, and deployment targets. These inputs need to be supplied in the right place.

Secrets should not be copied into source files or pasted into persistent instructions. Use the project's supported secret-management process. A setup command can ask which provider to use and request the necessary values, but it should avoid exposing secrets in logs or generated documentation.

When there is a manual action, identify it clearly. For example: create a database project, provide its connection string through the approved secret store, and run the project's deployment process. Do not say the setup is fully automatic if external credentials or account setup remain the developer's responsibility.

My own current template workflow is lightweight: clone, adapt project information, supply applicable environment variables, and deploy what is needed. The goal of future block composition is to make capability additions similarly straightforward within the supported combinations.

Move into Feature Work

Once the project identity, environment, and foundation are ready, discuss a real feature.

The implementation workflow can follow the process from earlier chapters:

1. The architect inspects the repository and researches the current technologies involved.
2. The proposal explains the user-visible behavior and the system areas affected.
3. A developer explores the code and raises remaining questions.
4. The architect resolves questions and updates the proposal.
5. The developer implements the agreed change on its own branch or worktree.
6. CI runs before review.
7. The reviewer examines the pull request, and the owner performs a final review.

The developer is not expected to be a product strategist. The foundation does not create the feature definition. The owner remains responsible for deciding what product behavior is wanted, while architecture translates that need into an understandable change.

The important benefit is that the feature discussion starts from a known base. When a new table, API operation, or application change appears in the proposal, it can be evaluated against existing conventions instead of being invented alongside initial setup.

A Deployed Base and a Valuable Product Are Different Milestones

A deployed foundation is an important first milestone because it shows that the prepared system can run in a real target environment. It is not proof that users have a reason to return.

The next milestone is a first feature that delivers recognizable value. Build a slice small enough to inspect but complete enough to test a real path. That could be creating an account, managing a core record, or completing a workflow that matters to the user.

Measure the first two wins separately: how quickly you reached a deployed starting point, and how quickly a useful feature worked on it. Those observations help determine whether the foundation saved time without confusing infrastructure speed with product success.

Keep the Product Decisions Connected to the Tasks

Product and architecture documents should not become static artifacts disconnected from implementation. If a question changes the behavior or affected boundaries, return to the person or agent responsible for the proposal. Make sure the developer implements the current decision, not a stale copy.

The exact editing and versioning process may differ from Git. In my workflow, internal documents are managed outside the source repository because they change during discovery and are not all useful context for future coding tasks. The current approved task still needs to be clear and available to the people or agents implementing and reviewing it.

If the feature reveals that a proposed boundary does not fit, stop and revise the decision. Continuing to code around a known mismatch only makes the eventual review more expensive.

Recognize What the Foundation Has Removed

After the first feature, look at the work you did not have to repeat. Did you avoid researching the same authentication integration? Did the dashboard call an API through an established client? Did the chosen block supply the relevant setup? Did the agent spend its time on the feature rather than discovering how the repository is organized?

Then identify what remained difficult. Maybe product rules were unclear. Maybe the deploy process required account setup. Maybe a capability was missing, or the agent still needed better context.

This separates the value of the foundation from the quality of feature discovery and implementation. If a task remains slow because its product behavior is undecided, adding more templates will not solve it. If the same integration issue appears across multiple projects, that may be a candidate for a reusable capability.

A good foundation does not replace the work of building a product. It gives that work a prepared place to begin.

Chapter 10: Evolving the Product Without Rebuilding Its Foundations

A prepared starting point is useful only if it keeps helping after launch. The product will change. Providers may change. New applications may be added. A foundation should make those changes easier to approach without requiring the team to recreate every integration.

The word maintainability can describe many properties, including readability, performance, testability, extensibility, and operational stability. The signal I watch most closely is practical: how long does it take to develop a feature? If comparable work that once took about a week begins taking two months, the project has probably accumulated friction that deserves investigation.

This is not a perfect metric. Features differ in scope, uncertainty, and product complexity. Treat development time as a signal to examine alongside examples of recent work, rather than as a standalone score.

Speed Matters After Startup

Speed is not only the time required to create a repository. It is the time and attention required to make successive changes safely.

A template can save a day of setup and still be a poor foundation if every feature requires a structural rewrite. A block can be easy to add but make API changes spread across every application. A generated project can be stable today and difficult to receive fixes from its source tomorrow.

To evaluate whether reuse continues to help, compare meaningful work across projects or across time:

- How much time went into deciding and integrating common capabilities?
- How many packages or applications changed for a feature?
- How often did implementation return to product or architecture because an important decision was missing?
- How much review was required to understand the scope and quality of the change?
- How frequently did a change in one capability force unrelated consumers to change?
- How long did a comparable feature take before and after adopting the foundation?

These observations do not reduce engineering quality to a single number. They help reveal where the project is becoming more expensive to change.

Keep the Advantage After the First Deployment

The initial benefit in my own experience was straightforward: after I had built and reused the SaaS template, I could start working on the product sooner. I was not rebuilding the database, authentication, API, and application structure from scratch.

That advantage depended on more than the files in the template. The applications and shared packages already had clear responsibilities. Applications consumed the server capabilities through

clients. The repository had conventions. The agent had a place to start and a base that already worked for my needs.

As the product grows, those boundaries become more important. A dashboard, public website, documentation app, command-line tool, SDK, or mobile client may all be separate consumers of shared capabilities. They do not all need to exist in the first release. But when one becomes necessary, a clean boundary can make it possible to add without rewriting the product around the first application.

Recognize a Reusable Capability

Not every feature belongs in a reusable block. My rule is based on experience while building: when I notice a capability that would be useful across several projects, I consider turning it into a block.

That decision should come after understanding the capability in at least one real product. Extracting too early can freeze an abstraction before its actual consumers are understood. Waiting until several projects have independently implemented the same capability can create duplicated work and inconsistent behavior.

A practical extraction discussion asks:

- Which behavior is common across the projects?
- Which parts vary by provider, product, or deployment?
- What contract would let consumers use the capability without knowing its implementation?
- Which dependencies are truly required?
- What configuration is needed, and when should the user be asked for it?
- Which project-specific assumptions should stay outside the block?

The answer may be a small shared package, a template convention, a block, or no reusable artifact at all. The goal is to preserve useful work, not to maximize the number of abstractions.

The Catalog Can Offer Different Implementations

Some capability types may eventually have several blocks. Authentication is a clear example: multiple libraries can provide broadly similar capabilities with different flows, storage expectations, and deployment assumptions.

That does not mean all providers are interchangeable. A catalog can make it easier for an agent to search, inspect descriptions, compare dependencies, and propose a suitable option. The developer still needs to consider the product's requirements and the actual provider documentation.

A useful catalog should help answer: what does this block provide, what does it require, what does it change, and under what conditions is it supported? The type tells us what role it claims to play. The details determine whether it fits this project.

A Block Has a Lifecycle

A reusable capability should be considered across its lifecycle:

- **Add** it to a compatible project.
- **Configure** it with the choices or credentials it requires.
- **Update** it when the block or its underlying dependencies change.
- **Replace** it when a different provider better fits the project.

- **Remove** it when the product no longer needs it.

My system is still developing. The intended system should support these lifecycle operations, but they are not all being presented here as already available DeesseJS features.

Add is the easiest operation to describe. Update, replace, and remove require understanding the local changes and dependencies around the block. Removal may affect another block that depends on it. Replacement may alter user models, APIs, or data formats. Updating a template can create conflicts with project-specific changes.

These operations should therefore make their effects visible and give the developer a reviewable result. Automatic work is valuable when it is supported and inspectable. It should not hide destructive or incompatible consequences.

Template Improvements and Local Projects

A template maintainer can improve a foundation after other projects have been created from it. The challenge is preserving a useful relationship between the original template and the customized project.

Copier documents a workflow that can update a generated project from a newer tagged version of its template, use prior answers, and surface conflicts that require manual review. This shows one way to support upstream evolution. It does not make arbitrary changes conflict-free, and it should not be confused with the DeesseJS block system. Copier, Updating a project

For a future system, updating an existing project from an improved foundation would be valuable. A project may have changed substantially since its initial clone, so any update mechanism needs to identify the upstream change, preserve local work where possible, and make conflicts understandable. This remains a product need to design, not a capability claimed as complete in this playbook.

When a foundation cannot be updated automatically, there are still options: apply a targeted fix manually, add a block to the local project, or start a new project from the improved template when that is appropriate. The best choice depends on the size and state of the project.

Parallel Agents Follow the Development Process

Each developer agent in my workflow has its own environment. It works on a branch or worktree and submits a pull request. If another change has moved the base forward, the agent can fetch that work, resolve conflicts, run checks, and update its branch. I review what is merged.

This is ordinary collaborative development, with agents performing developer tasks. Isolation matters because a shared main branch or shared working tree would allow one task to affect another before review. Pull requests provide a visible boundary for changes, while CI checks run before I spend time examining the implementation.

Reviewable work does not have to satisfy a universal line-count threshold. Focused changes are useful when they let a reviewer understand the intent, affected components, and evidence without reconstructing unrelated work. But breaking one coherent change into many artificial PRs can create overhead.

The evidence on size and review speed is not a simple rule. An empirical study of 845,316 pull requests across 100 active GitHub projects found no relationship between PR size or composition and time to merge, with additional data from Gerrit and Phabricator confirming the result. Kudrjavets,

Nagappan, and Rastogi, 2022 Focused scope still helps explain what a change intends to do, but smaller size alone is not a guarantee of faster review or better quality.

In my own process, a branch that has been under active development for more than a week is a warning signal, not a formal cutoff. It may indicate that the scope needs to be split, the proposal is incomplete, the branch is blocked, or the work has become more complex than expected. The right response is to inspect the cause.

Observe the Cost of Change

When a project starts feeling harder to maintain, compare actual feature work rather than relying on a vague sense that the code is messy.

Choose two or more features with similar scope. Record the time spent clarifying the need, investigating the architecture, implementing, reviewing, and correcting. Note how many components had to change and why. Was the work slow because of product uncertainty, a missing integration, unexpected coupling, poor context, or test failures?

This is not a benchmark unless the tasks are sufficiently comparable. It is a way to locate friction. If the product needs radically different features, longer development time may reflect higher complexity rather than declining maintainability.

The aim is to find recurring causes. If agents repeatedly need to understand the same API convention, improve the foundation or guidance. If an application needs database access because the API lacks a required operation, identify the server capability that is missing. If every new client duplicates authentication logic, inspect the shared contracts and clients.

Retain What the Work Teaches You

A defect becomes useful experience only if it changes what the team does next time.

When a failure repeats, identify whether it arose from a missing product decision, a stale technology assumption, an unclear boundary, a weak test, or insufficient isolation. Then retain the lesson in the place that can prevent a recurrence.

Some lessons belong in code: a test, lint rule, package boundary, or CI check. Others belong in concise contributor guidance or a current architecture decision. Historical discussions that are no longer relevant to implementation can stay in the external design workspace rather than the code repository.

This is an area I have not yet solved completely. I often fix a problem without reliably recording the lesson. I want a system where recurring lessons become durable protections, but I do not want to add large amounts of internal, volatile prose to every repository. The next step is to distinguish rules that can be enforced from decisions that require human or architect judgment.

Assumptions That Make Agent Work Fragile

The failures I have found most costly come from treating an agent as if it always has current, complete knowledge.

1. **Assuming its knowledge is current.** Models may use APIs or patterns from older versions. Check the installed version and current primary documentation.
2. **Assuming it knows every part of a technology.** A familiar name does not prove that it understands the specific behavior or integration your project uses.

3. **Giving it no boundaries.** Isolated environments, protected branches, limited credentials, and CI reduce the damage a mistaken command can cause.
4. **Keeping one conversation open indefinitely.** Long sessions can accumulate stale or conflicting context. Start a focused session when the task or decisions change substantially.
5. **Leaving consequential instructions implicit.** State expected behavior, constraints, current versions, and the relevant repository guidance.
6. **Treating disagreement as disobedience.** If an agent identifies a real contradiction, ask it to explain the evidence. The task or architecture may need revision.

These rules do not make the agent infallible. They make the work more bounded and the failures easier to recover from.

Reuse Should Preserve the Ability to Change

A foundation succeeds when it removes repeated setup while leaving the product understandable and adaptable. A block succeeds when it contributes a useful capability through a boundary that other components can rely on. A workflow succeeds when unclear decisions return to the right owner and implementation remains reviewable.

Reuse is not the absence of future work. It is work performed once where that work can serve more than one product, with enough information and checks to preserve its value.

The measure is not how much infrastructure the system contains. It is whether a new product can reach deployment and deliver its first useful feature faster, while later changes remain manageable.

Conclusion: From Understanding to Your First Project

The problem this playbook addresses is not that agents cannot produce software. They can produce applications that run and features that work. The harder question is whether the project remains understandable and changeable after the first version.

My own answer began with foundations. I spent months researching and improving a SaaS template, then discovered that reusing it changed the work I could do. I spent less time reopening the same integration questions and more time discussing the product. That experience led me to build DeesseJS: a system intended to preserve common foundations and make selected capabilities reusable.

The approach is still evolving. I do not consider my workflow complete, and some of the block lifecycle and update capabilities described here are future requirements. The useful next step is to apply the framework to one real project, observe what happens, and improve the parts that create friction.

A Framework You Can Apply

The chapters have developed one connected idea:

- A feature request activates product, technical, architecture, and verification decisions.
- Technology choices alone do not explain how their integrations fit together.
- Shared applications and packages need clear responsibilities and contracts.
- Optional blocks can provide capabilities without making every capability mandatory in every project.
- Agents need context appropriate to their role, plus access to current technical sources.
- Foundations and feature changes both need verification that matches the claims they make.
- Reuse should reduce future effort without hiding assumptions or blocking evolution.

The framework is not a maturity score. A project does not become maintainable by checking every box in a table. The point is to make the decisions that matter visible, then put each one in the right place: a product decision, an architecture boundary, a reusable capability, a test, or a review.

Diagnose the Project You Want to Build

Choose one new SaaS or web application you want to build. Use the following questions to identify where work is still being left implicit. For each answer, write the evidence that supports it. If you do not know, mark it as an open decision.

Area	Question	Useful evidence
Product	What should the first version enable for its users?	A concrete behavior and a defined scope

Area	Question	Useful evidence
Knowledge	Which technical choices still rely on assumptions?	Current documentation and explicit uncertainties
Foundations	Which recurring capabilities are already prepared?	Working integrations with known support conditions
Composition	Which additional capabilities does the product require?	A justified selection and its dependencies
Interfaces	How do applications consume shared capabilities?	Defined contracts and responsibility boundaries
Roles	Who decides, implements, and reviews?	A workflow and access appropriate to each role
Context	What does the developer need for its assigned task?	Current instructions and scoped references
Verification	What establishes that the relevant behavior works?	Checks and observations tied to the behavior
Isolation	Which systems can the developer agent change?	Actual environment and permission boundaries
Evolution	What could make future changes expensive?	Known coupling, uncertainties, and comparable work

The purpose is to identify where work is implicit, not to resolve every possible future problem. Focus on uncertainties that affect the first result and the foundations it depends on.

Reach a Deployed Starting Point

First, choose a foundation that fits the application type and its required capabilities. Adapt its identity, provide the supported setup with the inputs it requires, and deploy the base.

This first milestone makes the starting point concrete. It does not establish that the product solves a user's problem. The next step is a product-specific feature that you can implement, inspect, and review.

You remain responsible for defining that feature. A foundation can provide authentication, data access, API conventions, and other recurring capabilities. It cannot decide what users need or whether the feature is valuable.

Start with One Useful Slice

Choose a feature with a recognizable result and manageable scope. Clarify its behavior. Identify the existing capabilities it should use and the applications or packages it affects. Capture the product and architecture decisions in a proposal that describes the impact, not a brittle sequence of file edits.

Let the developer inspect the repository and raise questions before implementation. If a question concerns product behavior, a dependency, a schema, or an architectural boundary, return it to the responsible decision maker. If the issue is local implementation detail, let the developer use its judgment and explain the choice in the pull request.

Use a separate working environment, a branch or worktree, CI checks, an independent review where useful, and your own final review. If the implementation reveals that the proposal is wrong, revise the decision rather than building around an unresolved contradiction.

The first feature tests more than the code. It tests whether the foundation, context, handoff, implementation, and verification fit together.

Where DeesseJS Enters the Workflow

DeesseJS is the system I am developing to make more of this preparation reusable.

Templates provide starting foundations. The block system is intended to let users and agents discover selected capabilities, understand their dependencies, and add supported combinations through a command that performs the integration and asks for needed inputs at the right point.

Different parts of the system are at different stages. The template workflow I use today is real. Broader automatic composition, block update and replacement, and propagating template improvements into existing projects are requirements or goals described in this playbook, not promises that every feature is already available.

Before using a foundation or block, inspect the current product and its documentation. Confirm what it supports, which providers it works with, what it changes, and what configuration remains. The playbook intentionally does not invent commands, prices, release guarantees, or a product link that has not been verified.

The invitation is practical: take one project you already want to build and determine whether an available DeesseJS foundation removes repeated work that project would otherwise require.

Know What Each Part Contributes

Templates and blocks provide reusable structure and supported integrations. Current references and conventions help agents work with the technologies in the repository. Conception translates a product need into explicit behavior and affected boundaries. Developers implement the agreed change. CI and tests catch defined regressions. Review examines the result against the intended behavior. Isolation limits the changes an agent can affect.

No single part replaces the others. A prepared foundation does not establish that a feature is correct. A test suite does not decide what the product should do. A detailed proposal does not make implementation quality automatic. The method works when each part covers the kind of uncertainty it is suited to cover.

Build the Foundations Yourself or Reuse Them Deliberately

You can build these foundations yourself. This playbook is intended to help you understand the decisions and effort involved, then evaluate whether the result fits your project.

You can also reuse work that already exists and fits your needs. That is why I am building DeesseJS: to preserve research, integrations, conventions, and corrections that would otherwise be repeated across projects.

The choice should be informed by the project and the capabilities currently available. A prepared integration can save recurring work. An unsuitable integration can introduce assumptions that need to be changed. Use the framework to inspect the difference before you depend on it.

Your Next Action

Take one product you want to build. Write down its first useful behavior and identify the foundation it needs. Inspect the available DeesseJS templates for a fit. If one fits, start from it, adapt the project identity, supply the required configuration, and deploy the base.

Then implement one product-specific feature through the process in this playbook. Record what became faster, what remained unclear, what the agents got wrong, and which checks caught problems. That evidence gives you a concrete basis for deciding what to improve next.

If a capability is missing or an integration needs work, make the gap explicit. Research it, decide whether it is specific to the product or reusable across projects, and retain any durable correction where future work can use it.

The change I want to help create is a different way to organize software development with agents: prepare recurring technical work, make consequential decisions explicit, assign bounded responsibilities, verify the result, and retain what you learn. Reusing that preparation can give more of your effort back to the product you intended to create.

Sources and Further Reading

- Microsoft Azure Well-Architected Framework, Architecture Decision Records
- AWS Prescriptive Guidance, ADR Process
- DORA, State of AI-assisted Software Development 2025
- Google Testing Blog, “In Praise of Small Pull Requests”
- Google Engineering Practices, “Small CLs”
- Kudrjavets, Nagappan, and Rastogi, 2022
- Arabat and Sayagh, “An empirical study on cross-component dependent changes: A case study on the components of OpenStack,” 2024
- shadcn/ui, Registry overview
- Registry item schema
- Namespaced registries
- SEI, Integrability
- Pact, Consumer testing
- Copier, Updating a project